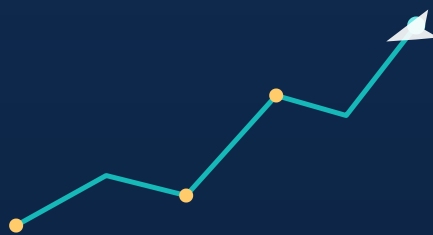


Como construir tu primera app con IA

Un metodo completo y reutilizable para transformar una idea en un producto real, documentado en tiempo real mientras se construia, paso a paso, sin omitir ningun detalle ni ningun error en el camino.

Caso de estudio: Somos-Rata — Flight Price Alert Web



Indice

Este libro documenta, capitulo a capitulo, el metodo completo para construir un producto real con ayuda de una IA como copiloto tecnico: desde la formulacion del problema hasta tener usuarios reales usando el producto y dando feedback. Cada capitulo se apoya en hechos reales del proyecto Somos-Rata, construido como caso de estudio en vivo.

00	Introduccion	Que quisimos construir y por que
01	De la idea al MVP	Como encontrar y acotar un problema real
02	La metodologia	El flujo de trabajo completo, paso a paso
03	Prompt engineering	Como hablarle a una IA para que sirva de verdad
04	Arquitectura del proyecto	Explicada para quien recién empieza
05	Trabajar con Claude Code	La mecanica real de colaboracion
06	Desarrollo del MVP	La historia completa, cronologica
07	Errores que cometimos	11 bugs reales, causa raiz y solucion
08	Habilidades desarrolladas	15 skills entrenadas construyendo el proyecto
09	Framework reutilizable	El metodo condensado en 9 fases
10	Checklist definitivo	226 puntos, de la idea a usuarios reales
11	Filosofia de trabajo	La forma de pensar detras del metodo
12	Lecciones finales	Que repetir, que cambiar, y un consejo final

00 Introduccion

Que quisimos construir, por que nacio la idea, y que aprendimos antes de escribir una sola linea de codigo.

Que queriamos construir

Queriamos construir una herramienta simple: una pagina web donde cualquier persona pudiera decir «quiero volar de Santiago a Madrid, en tal fecha, y no quiero pagar mas de tanto dinero», y que un sistema automatico se encargara de vigilar los precios reales del mercado y avisar en el momento en que ese precio —o uno mejor— apareciera.

En una frase: *«Dinos tu precio. Nosotros te avisamos»*. Ese eslogan, que terminamos usando literalmente como marca, resume exactamente el contrato de valor del producto. No es un buscador de vuelos —existen decenas, como Skyscanner o Google Flights—. Es un **vigilante de precios** que trabaja para ti mientras haces otra cosa.

Por que nacio la idea

La idea nace de una friccion real y muy comun: cuando alguien quiere viajar, generalmente no sabe el precio «correcto» de un vuelo. Entra a un buscador, ve un precio, no sabe si es caro o barato, y tiene dos opciones malas:

1. Comprar de inmediato por miedo a que suba —y quizas pagar de mas.
2. Esperar y revisar el precio manualmente todos los dias —perdiendo tiempo, y con el riesgo de que el precio bueno aparezca un dia que uno no revisa, y desaparezca antes de la proxima revision.

Este patron de comportamiento —revisar manualmente un precio que cambia en el tiempo, con la esperanza de «cazar» el momento correcto— se repite en muchisimos dominios ademas de los vuelos: entradas a conciertos, el dolar observado, precios de electrodomesticos en oferta, arriendos, criptomonedas. Cuando un problema tiene esta forma —«un valor cambia en el tiempo y quiero que alguien lo vigile por mi y me avise»—, es un buen candidato para automatizar con una alerta.

Cual era el problema, formulado de forma precisa

Es muy importante, antes de programar cualquier cosa, poder escribir el problema en una frase clara. Si no puedes hacer eso, todavia no tienes un problema bien definido: tienes una vaga inquietud. La formulacion que usamos fue:

PROBLEMA FORMULADO

«Las personas que quieren viajar no tienen tiempo ni ganas de revisar precios de vuelos manualmente todos los días, y por eso o compran caro por ansiedad, o se pierden ofertas por falta de seguimiento.»

Nota la estructura de esta frase, porque es un patron reutilizable:

[QUIEN] + [QUE NECESITA O LE FALTA] + [QUE CONSECUENCIA NEGATIVA TIENE ESO]

- **Quien:** personas que quieren viajar.
- **Que le falta:** tiempo o ganas para revisar precios manualmente.
- **Consecuencia:** pagan de mas por ansiedad, o pierden ofertas por no hacerle seguimiento.

Cuando puedas rellenar esta plantilla para tu proxima idea, ya tienes un problema formulado. Si no puedes, es senal de que necesitas investigar mas antes de construir nada.

Como validamos que era un problema real

Hay que ser honesto en este punto para que el manual sea util de verdad: en el caso de Somos-Rata, la validacion de «esto es un problema real» no fue un proceso formal de entrevistas de usuario ni encuestas de mercado. Fue mas simple y mas rapido: es un problema que cualquiera reconoce de inmediato al escucharlo —«si, a mi me ha pasado»— porque es una experiencia casi universal de quien ha comprado pasajes aereos alguna vez.

Esto es una validacion **debil pero suficiente** para justificar construir un MVP rapido —días, no meses— y ponerlo a prueba con personas reales. La validacion **fuerte** —la que de verdad importa— vino despues, cuando compartimos el link con un primer grupo de quince personas reales, y observamos:

- Si de verdad creaban alertas —uso real, no solo curiosidad.
- Que dudas tenian —feedback cualitativo real.
- Que errores encontraban —lo cual revela huecos en el producto.
- Si volvian a usarlo o no.

LA LECCION

No esperes tener una validacion perfecta antes de construir. Para problemas que son evidentemente comunes —todo el mundo entiende el dolor en cinco segundos—, es mas eficiente construir un MVP barato y validar con uso real, que gastar semanas en investigacion de mercado formal. Esta regla no aplica a todos los problemas: si el tuyo es de un nicho muy

especifico que tu mismo no vives, si necesitas investigacion previa mas seria —entrevistas, encuestas— antes de construir, porque el riesgo de estar equivocado es mucho mayor.

Que aprendimos antes de escribir una sola linea de codigo

Cuando la conversacion que dio origen a este manual comenzo, el proyecto ya existia en un estado inicial: un MVP en modo demo, corriendo en local, con datos simulados —*mock*— y guardado en archivos CSV en vez de una base de datos real. Es decir, el trabajo de diseno inicial —que campos pedir en el formulario, que logica de negocio aplicar, que arquitectura de codigo usar— ya se habia hecho antes de esta conversacion, en una sesion previa de trabajo con IA.

Esto revela un patron muy importante que vale la pena nombrar explicitamente:

FASE 0 (EXPLORATORIA, OFFLINE)

Disenar el modelo de datos, la logica de negocio y la arquitectura del codigo en un entorno conversacional donde se puede iterar rapido sobre **ideas** antes de comprometerse a **escribir codigo real**.

FASE 1 (DE EJECUCION, CON EL PROYECTO REAL)

Ya con un esqueleto de codigo funcionando en modo demo, pasar a conectar todo lo real: base de datos en la nube, repositorio de codigo, hosting publico, proveedor de emails, proveedor de precios de vuelos, WhatsApp.

Este libro documenta enteramente la Fase 1: pasar de «*funciona en mi computador con datos falsos*» a «*funciona en internet con datos reales y usuarios reales*». Esa transicion —de demo local a producto en produccion— es el nucleo de lo que este libro te va a enseñar a repetir.

Lo que ya sabiamos, sin haberlo programado nosotros mismos linea por linea, gracias a ese trabajo previo:

- Que el modelo de datos necesitaba cuatro tablas: usuarios, alertas, historico de precios, notificaciones.
- Que la logica de negocio necesitaba dos reglas separadas: un umbral de precio, y una ventana anti-spam —no notificar la misma alerta mas de una vez cada cierta cantidad de horas.
- Que la arquitectura debia separar la web publica —donde la gente crea alertas— del *worker* que revisa precios —un script independiente que corre en un horario fijo, no cada vez que alguien visita la pagina.

- Que el sistema debia funcionar en modo demo sin ninguna credencial real, y pasar a modo real solo cambiando variables de entorno, sin reescribir codigo.

Este ultimo punto —diseñar el sistema para que funcione en modo demo y en modo real con el mismo codigo, solo cambiando configuracion— es una decision de arquitectura extremadamente valiosa, que se explica en profundidad en el Capitulo 04. Es la razon por la cual pudimos avanzar tan rapido: nunca tuvimos que «convertir» el proyecto de demo a real, simplemente fuimos activando piezas reales una por una.

Que vas a encontrar en el resto de este libro

El resto de este libro documenta, con el maximo detalle posible, todo lo que paso desde que se abrio esta conversacion —con un MVP funcionando en modo demo— hasta que el producto quedo:

- Desplegado publicamente en internet —Streamlit Community Cloud.
- Con una base de datos real en la nube —Supabase / PostgreSQL.
- Enviando emails reales —SendGrid.
- Buscando precios de vuelos reales —Duffel, tras descartar Amadeus.
- Enviando WhatsApp reales —Twilio Sandbox.
- Con identidad de marca propia —Somos-Rata: logo, colores, tono.
- Probado con quince o mas usuarios reales, que generaron alertas de verdad.
- Con multiples bugs reales encontrados y corregidos en produccion, mientras usuarios reales lo estaban usando.
- Con una funcionalidad nueva agregada a mitad del beta —buscar por nombre de ciudad en vez de codigo IATA— en respuesta a feedback real.

Todo esto en menos de una semana de trabajo. Ese ritmo no es casualidad: es el resultado directo de la metodologia que se explica en el proximo capitulo.

01 De la idea al MVP

Teoría de producto aplicable a cualquier idea futura, no solo a Somos-Rata: como encontrar un problema, validarlo, acotar el alcance mínimo, y priorizar sin sobre-construir.

Que es un MVP —y que no es—

MVP significa *Minimum Viable Product*, Producto Mínimo Viable. La palabra clave que casi todo el mundo malinterpreta es «Viable», no «Mínimo». Un MVP no es «lo más chico y feo que puedo construir». Un MVP es lo mínimo que necesitas construir para que el producto sea **viable**, es decir, que de verdad resuelva el problema para alguien de principio a fin, aunque sea de forma simple.

La versión mínima de Somos-Rata que resuelve el problema completo —no una versión a medias— necesitaba, como mínimo:

1. Un lugar donde el usuario diga que quiere —formulario.
2. Un lugar donde eso quede guardado —base de datos.
3. Un proceso que revise precios reales, no simulados, periódicamente.
4. Una forma de avisarle al usuario cuando corresponde —notificación real.

Si falta cualquiera de esos cuatro puntos, el producto no es viable: no resuelve el problema de punta a punta. Notaras que esta lista **no incluye**, por ejemplo, cuentas de usuario con login, un dashboard bonito, soporte para diez monedas distintas, recomendaciones con inteligencia artificial de «cuando comprar», o integración con cinco proveedores de vuelos distintos. Todo eso sería valioso en una versión 2.0, pero ninguno de esos puntos es necesario para que el producto sea viable la primera vez.

LA DISCIPLINA DE UN MVP

Cada cosa que agregas, preguntate: «si esto no estuviera, el producto dejaría de resolver el problema completo?». Si la respuesta es no, no es parte del MVP.

Como encontrar un problema

Fuentes típicas de buenos problemas, ordenadas de más confiable a menos confiable:

- **Tu propia frustración repetida.** Si a ti te ha pasado algo más de una vez y cada vez pensaste «esto debería ser más fácil», ahí hay una señal fuerte. Ya eres tu propio primer usuario y entiendes el

contexto sin tener que investigarlo.

- **Frustraciones que escuchas en otros, repetidas.** Si mas de una persona, sin conocerse entre si, se queja de lo mismo de forma independiente, hay senal de que el problema es comun, no un caso aislado tuyo.
- **Tareas manuales y repetitivas que alguien hace «a mano».** Cualquier tarea que un humano hace de forma repetitiva y mecanica —revisar un precio todos los dias, copiar datos de un lado a otro, generar el mismo reporte cada semana— es candidata a automatizacion.
- **Patrones «vigilar y avisar».** Cualquier problema con la forma «un valor cambia en el tiempo y quiero que alguien lo vigile por mi» es un patron de producto muy conocido y replicable —alertas de precio, de stock, de tipo de cambio, de disponibilidad. Somos-Rata es una instancia de este patron aplicado a vuelos.

Como validar si vale la pena resolverlo

Antes de construir, hazte estas preguntas —adaptadas del metodo real que seguimos, mas honesto sobre lo que si hicimos y lo que es la version ideal del proceso—:

PREGUNTA 1: PUEDO EXPLICAR EL PROBLEMA EN UNA FRASE QUE CUALQUIERA ENTIENDA EN CINCO SEGUNDOS

Si la respuesta es si —como en Somos-Rata: «revisar precios de vuelos manualmente es una lata y por eso la gente pierde ofertas o compra caro»—, es una buena senal: el problema es intuitivo, no necesitas convencer a nadie de que existe. Si necesitas tres parrafos para explicarlo, es senal de alerta: puede que sea muy de nicho, o que en realidad no sea un problema tan real, sino una solucion buscando un problema.

PREGUNTA 2: EXISTEN ALTERNATIVAS HOY, Y POR QUE NO SON SUFICIENTES

Para Somos-Rata: si, existen buscadores de vuelos —Google Flights, Skyscanner—. La diferencia es que ellos requieren que tu revises manualmente; nuestra propuesta es que el sistema te avise a ti, sin que tengas que volver a revisar. Es una diferencia de mecanismo —push versus pull—, no de datos. Si tu idea no tiene ninguna ventaja de mecanismo, forma o enfoque distinto a lo que ya existe, es dificil justificar por que alguien cambiaria de habito para usar tu producto.

PREGUNTA 3 (LA IDEAL, QUE EN ESTE PROYECTO SE SALTO POR VELOCIDAD)

¿Puedo validar esto hablando con cinco o diez personas reales **antes** de construir nada, para confirmar que de verdad les pasa y que pagarian o usarian una solucion? Esto es lo que un proceso de validacion «de manual de libro» te recomendaria. En Somos-Rata no se hizo asi: se construyo directo un MVP funcional y se valido despues, con el producto ya construido, en un grupo de quince personas reales.

Esto funciona porque el problema es de validación débil-suficiente: es tan común y fácil de entender que el riesgo de estar equivocado es bajo. Si tu problema es más de nicho o más dudoso, si deberías hacer esta validación antes.

Como definir el alcance de un MVP sin construir de mas

Técnica práctica: escribe la lista de todo lo que se te ocurra que el producto podría tener. Después, para cada ítem, clasifica en tres columnas:

Columna A — MVP	Columna B — Deseable	Columna C — Descartado
Sin esto, el producto no resuelve el problema de punta a punta.	Mejora la experiencia, pero el producto funciona sin esto.	Idea interesante, sin relación directa con el problema central.

Ejemplo real aplicado a Somos-Rata:

- **Columna A —si, MVP:** formulario de alerta, base de datos, bot que revise precios reales periódicamente, envío de notificación —email como mínimo, WhatsApp como plus—, lógica anti-spam.
- **Columna B —deseable, agregado después por feedback real:** buscar por nombre de ciudad en vez de código IATA —se agrega recién cuando usuarios reales del beta reportaron la fricción—; dashboard administrativo con KPIs; identidad de marca completa.
- **Columna C —descartado o pospuesto indefinidamente:** soporte multi-moneda real con conversión de divisas; migración a WhatsApp Business API; predicción de precios con machine learning.

Como evitar construir demasiado

El riesgo más común al construir un MVP **no** es que te quede muy simple. Es que te quede demasiado complejo porque no supiste decir que no a tiempo. Síntomas de que te estás pasando de MVP:

- Estás construyendo un sistema de login o cuentas de usuario «por si acaso», antes de saber si alguien va a usar el producto más de una vez.
- Estás soportando cinco proveedores distintos de un mismo servicio cuando con uno alcanza para validar.
- Estás invirtiendo tiempo en un panel de administración elaborado antes de tener un solo usuario real.

- Estas bloqueado por una decision de arquitectura «perfecta» en vez de avanzar con la opcion mas simple que funcione hoy.

CONTRA EJEMPLO REAL

El codigo de Somos-Rata tiene, desde el inicio, dos formas de guardar datos —un backend de CSV local, y uno de Supabase/Postgres en la nube—, intercambiables con una sola variable de entorno. Suena a sobre-ingenieria, pero es lo opuesto: fue la decision que permitio desarrollar y probar todo el sistema sin depender de internet ni de una base de datos real, y cuando llego el momento de ir a produccion, cambiar a la version real fue instantaneo, sin reescribir nada.

Como priorizar

La regla que se siguio, implicita en todo el proyecto, fue: **prioriza en el orden en que una pieza depende de la anterior para tener sentido, y dentro de eso, prioriza lo que reduce mas incertidumbre primero**. El orden real seguido en el proyecto:

1. Base de datos real —sin esto, nada de lo demas puede persistir datos reales.
2. Control de versiones y repositorio —sin esto, no hay forma segura de desplegar.
3. Despliegue publico —sin esto, no hay producto que mostrarle a nadie.
4. Email real —el canal de notificacion mas confiable y universal.
5. Precios reales —sin datos reales, las alertas nunca podrian dispararse de verdad.
6. WhatsApp —canal adicional, mas complejo de configurar, por eso al final.
7. Identidad de marca —una vez que el sistema funciona de punta a punta.
8. Mejoras basadas en feedback real —solo despues de tener usuarios reales.

Como pensar como fundador, no como programador

Esta es quizas la diferencia mas importante entre «hacer un proyecto de programacion» y «construir una startup», aunque el codigo escrito sea identico. Un programador pregunta: «¿como implemento esto?». Un fundador pregunta primero: «¿vale la pena implementar esto? ¿Que pasa si no lo hago? ¿Que estoy aprendiendo al hacerlo?».

Tres ejemplos reales donde el pensamiento de fundador se impuso sobre el pensamiento puramente tecnico:

AMADEUS VERSUS DUFFEL

Cuando se descubrió que el portal gratuito de Amadeus estaba por cerrarse, un pensamiento puramente técnico habría dicho «hay que arreglar el registro como sea». El pensamiento de fundador preguntó: «¿realmente necesito ese proveedor específico, o necesito un proveedor de precios de vuelos que funcione?». Eso llevó a pivotar rápidamente a Duffel.

LA MONEDA CLP

La solución técnica «correcta» habría sido construir un sistema de conversión de monedas en tiempo real. La decisión de fundador fue distinta: limitar temporalmente el producto a EUR, comunicarlo con claridad, y dejar la conversión real como mejora futura. Esto prioriza «no mentirle al usuario con datos incorrectos» por sobre «tener la funcionalidad completa inmediatamente».

CUANTAS PERSONAS INVITAR

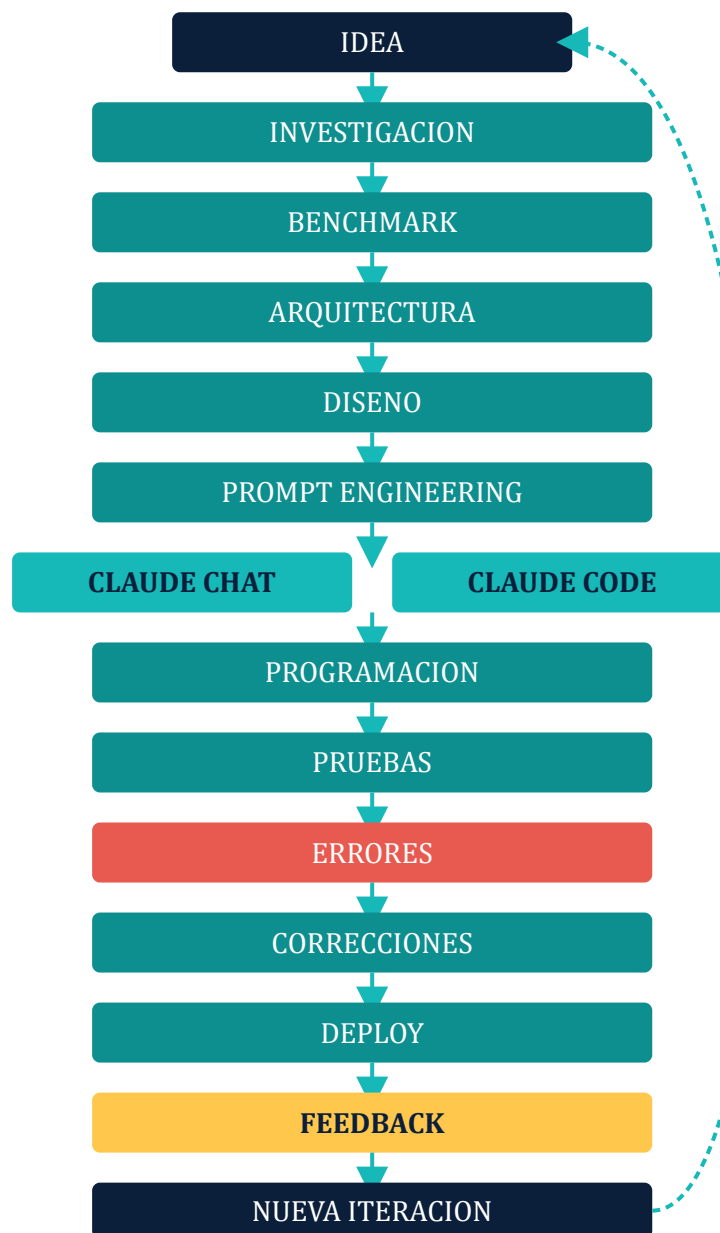
Antes de compartir el link público, se hizo la pregunta de fundador «¿mis proveedores gratuitos aguantan esto sin romperse ni costarme dinero inesperado?» antes de simplemente compartir el link sin pensarlo.

Pensar como fundador es preguntarse constantemente «¿por qué estoy haciendo esto, y qué pasa si no lo hago?», no solo «¿cómo lo hago técnicamente?». La IA puede ayudarte con el «cómo» casi siempre. El «por qué» y el «vale la pena» siguen siendo, y deben seguir siendo, decisiones tuyas.

02 La metodología que seguimos

El flujo de trabajo completo, de la idea al usuario real, explicado paso a paso para quien no tiene experiencia previa.

El diagrama general



El ciclo completo. Es un círculo, no una línea recta: se recorre varias veces por proyecto, cada vuelta mas rápida que la anterior.

Cada etapa explicada para principiantes

IDEA

Ya cubierto en el capítulo anterior: es el punto de partida, un problema formulado en una frase.

INVESTIGACION

Antes de construir, entender el contexto: ¿quién más resuelve esto? ¿cómo lo resuelven? ¿qué le falta a esas soluciones? En Somos-Rata, esto significó entender que existen buscadores de vuelos, pero ninguno hace «vigilancia pasiva con notificación» de forma simple y gratuita, centrada en ese flujo específico.

BENCHMARK

Comparar opciones técnicas concretas para cada pieza que tu sistema necesita. No se trata de elegir «la mejor tecnología del mundo», se trata de elegir la que resuelve tu necesidad específica con el menor costo y fricción. Para hosting se eligió Streamlit Community Cloud —gratis, integrado con GitHub, ideal para un MVP en Python—. Para base de datos, Supabase —Postgres gratis en la nube, con panel visual—. Para precios de vuelos, se evaluó Amadeus primero, se descubrió que su portal gratuito se discontinuaba, y se hizo un benchmark en vivo comparando la alternativa —Duffel— antes de decidir el cambio.

ARQUITECTURA

Decidir que piezas existen, que hace cada una, y cómo se comunican entre sí, antes de escribir el código detallado de cada una. Se explica en profundidad en el Capítulo 04.

DISEÑO

En este proyecto, el diseño de interfaz fue deliberadamente minimalista al principio, y se invirtió en una identidad de marca recién después de tener el sistema funcionando técnicamente de punta a punta. El diseño visual es la parte más fácil de cambiar después sin romper nada; la arquitectura y la lógica de negocio son las más difíciles de cambiar después.

PROMPT ENGINEERING

La habilidad de traducir una necesidad de negocio o técnica en instrucciones que una IA pueda ejecutar bien. Se explica en detalle en el Capítulo 03.

CLAUDE CHAT VERSUS CLAUDE CODE

Claude Chat es para explorar ideas, discutir arquitectura, pedir explicaciones, sin que la IA toque tu proyecto real. Claude Code es para ejecutar: tiene acceso directo a tus archivos, puede correr comandos

en tu terminal, instalar cosas, hacer commits, y de hecho hizo todo eso durante esta conversacion. Se explica en profundidad en el Capitulo 05.

PROGRAMACION

El momento en que las decisiones se convierten en archivos reales. En este proyecto, la mayoría del «programar» fue realizado directamente por Claude Code bajo la direccion del usuario, quien tomaba las decisiones de producto y aprobaba los cambios. Esto es una forma legitima y cada vez mas comun de «programar» en la era de la IA: dirigir en vez de teclear.

PRUEBAS

Un patron que se repitio constantemente: despues de cada cambio, antes de darlo por terminado, se verificaba con una prueba real —no solo «revisar que el codigo se vea bien»—. Enviar un email de verdad y pedirle al usuario que confirme si llego; enviar un WhatsApp de verdad; correr el proceso de revision de precios manualmente y leer el resultado linea por linea. Esta disciplina de **verificar con la cosa real, no asumir** es probablemente la practica individual mas importante de toda la metodologia.

ERRORES

Van a pasar. La actitud correcta no es evitarlos a toda costa —imposible—, sino: detectarlos rapido con pruebas reales; diagnosticar la causa raiz, no solo el sintoma superficial; corregir la causa, no poner un parche que oculte el sintoma. Todo el Capitulo 07 esta dedicado a esto.

CORRECCIONES

Una vez identificada la causa, corregirla implica frecuentemente entender bien por que paso, para que la correccion prevenga que vuelva a pasar, no solo que arregle el caso puntual.

DEPLOY

En este proyecto, el despliegue fue automatico: cada vez que se hacia **git push**, Streamlit Community Cloud redespelgaba la web publica sola, en uno o dos minutos. Esto se llama **despliegue continuo**, y es una de las razones por las que se pudo iterar tan rapido.

FEEDBACK

Una vez que hubo usuarios reales, el feedback empezo a llegar en forma de preguntas directas, reportes de comportamiento raro, y observacion directa de los datos guardados en la base de datos.

NUEVA ITERACION

Cada vuelta del ciclo termina generando una nueva idea o mejora, que vuelve a empezar el ciclo desde el principio, pero mas rapido, porque la base ya existe. El ejemplo mas claro es la funcionalidad de

«buscar por nombre de ciudad»: nacio directamente de feedback real de usuarios confundidos con codigos IATA, y paso por el ciclo completo en un tiempo mucho menor que las integraciones originales.

Una variante importante: el «modo plan» antes de programar

En dos momentos especificos de este proyecto —la funcionalidad de busqueda por ciudad, y la ampliacion del diccionario de ciudades—, se pidio explicitamente: «antes de tocar codigo, muéstrame el plan». Esto activa un modo de trabajo distinto: en vez de pasar directo de *prompt engineering* a *programacion*, se inserta un paso intermedio de diseno explicito y aprobacion, donde la IA:

1. Explora el codigo existente para entender el contexto real —no asume.
2. Propone un plan detallado por escrito: que archivos va a tocar, que funciones va a crear, que decisiones de diseno esta tomando y por que.
3. Si hay ambigüedades o decisiones con mas de una opcion razonable, las pregunta explicitamente en vez de decidir solo.
4. Espera aprobacion explicita del humano antes de escribir una sola linea de codigo.

REGLA PRACTICA

Esta practica es extremadamente recomendable para cualquier cambio de tamano medio o grande —mas de dos o tres archivos, o que involucre una decision de diseno no trivial—. Para cambios chicos y obvios, no hace falta este paso. Si al leer el pedido no tienes cien por ciento claro como se deberia implementar sin ambigüedad, pide el plan primero.

Otra variante: verificacion de datos antes de usarlos

Un patron que aparecio en la ampliacion del diccionario de ciudades merece mencion aparte: cuando el usuario le paso a la IA una lista de codigos IATA para agregar al sistema, y uno de esos codigos —WCA para Castro/Chiloe— resulto estar desactualizado —correspondia a un aeropuerto que ya no opera vuelos comerciales—, la IA no lo agrego ciegamente: lo verifiko con busquedas independientes antes de escribirlo en el codigo, y encontro que el codigo correcto y vigente era otro —MHC—.

La leccion generalizable: cuando trabajas con datos que tienen un costo de estar mal —un codigo de aeropuerto incorrecto rompe la busqueda de vuelos para esa ciudad, silenciosamente, sin ningun error visible—, pide explicitamente que se verifiquen contra una fuente antes de usarlos, en vez de confiar en la memoria de la IA o en tu propia memoria. «Prefiero que me preguntes por uno dudoso a que pongas uno incorrecto» fue literalmente la instruccion que se dio, y es una gran instruccion general para cualquier dato critico.

03 Prompt engineering

Como hablarle a una IA para que te sirva de verdad. Ejemplos reales de prompts que funcionaron, y de prompts que necesitaron una vuelta extra de aclaracion.

La idea central

«Prompt engineering» suena tecnico, pero en el fondo es una habilidad de **comunicacion**, no de programacion. Es la diferencia entre pedirle algo a un colega nuevo que no conoce el contexto de tu proyecto, versus pedirselo a alguien que lleva un ano trabajando contigo. Un buen prompt le da a la IA el contexto que necesita para tomar buenas decisiones sin que tengas que supervisar cada detalle.

La calidad de lo que obtienes de una IA es directamente proporcional a la calidad del contexto y la claridad de la instruccion que le das. Esto no es una frase motivacional, es un hecho observable en este proyecto: cada vez que el prompt fue vago o incompleto, la respuesta necesito una vuelta extra de aclaracion; cada vez que el prompt fue especifico y con contexto, la ejecucion fue directa y correcta a la primera.

Prompts que funcionaron bien

EJEMPLO 1 — EL PROMPT INICIAL QUE ARRANCO TODO EL PROYECTO

```
«Este es mi proyecto Flight Price Alert Web, un MVP en Streamlit + Python que ya corre en local en modo demo (mock + CSV). Quiero desplegarlo como web publica con link compartible para que cualquiera cree alertas desde el celular. Ayudame paso a paso en este orden: (1) configurar Supabase... (2) subir el repo a GitHub, (3) desplegar en Streamlit Community Cloud... (4) conectar SendGrid... (5) conectar Amadeus... (6) Twilio WhatsApp al final.»
```

Por que funciono tan bien:

- Da el estado actual exacto —«ya corre en local en modo demo».
- Da el objetivo final claro —«web publica con link compartible».
- Da el orden de ejecucion deseado, numerado, sin ambigüedad.
- No pide «hazme una app de alertas de vuelos» desde cero, lo cual habria obligado a la IA a rediseñar algo que ya existia y ya estaba bien pensado.

EJEMPLO 2 — PEDIR UN PLAN ANTES DE PROGRAMAR

«Antes de tocar código, muéstrame el plan. Cambios que necesito: [lista detallada de requisitos técnicos y de negocio]... Después de que apruebe el plan: aplica los cambios, pruébalos en local... y ayúdame a hacer commit y push.»

Por que funciona tan bien:

- Separa explícitamente diseño de ejecución, evitando que la IA empiece a escribir código antes de que ambos estén de acuerdo en el enfoque.
- Incluye contexto técnico verificado por el propio usuario de antemano —«Duffel acepta códigos de CIUDAD IATA, no solo códigos de aeropuerto»—, lo cual evita que la IA tenga que investigar desde cero.
- Cierra con criterios de éxito claros —«prueba que Madrid resuelva a MAD y París a PAR».

EJEMPLO 3 — REPORTAR UN BUG CON DATOS CONCRETOS, NO UNA SENSACION VAGA

«Duda: los usuarios están colocando valores en USD y en CLP, por ejemplo uno puso MAD - SCL en 380 USD. pero hay otro que puso en CLP 500 de ARI a SCL, pero seguro se equivocó. dime si los valores lo está tomando bien el buscador en precio CLP o solo debería buscar en USD.»

Por que funciona bien —aunque no sea un prompt «perfecto» en forma—:

- Da ejemplos concretos de datos reales observados, no una queja abstracta como «algo está mal con las alertas».
- Formula una hipótesis propia —«seguro se equivocó»— sin insistir en ella como si fuera un hecho, dejando espacio para que la IA investigue y la corrija si hace falta —de hecho, la investigación reveló que había un bug real, no solo un error de usuario.
- Esto llevó a que la IA fuera a mirar los datos reales en la base de datos en vez de especular, y encontrara el problema real: comparar precios en monedas distintas sin conversión.

EJEMPLO 4 — PEDIR VERIFICACION EN VEZ DE CONFIAR EN LA MEMORIA

«Verifica que cada código IATA sea el correcto antes de agregarlo -- si dudas de alguno, avísame en vez de inventarlo.»

Por que funciona bien: instruye explícitamente el comportamiento deseado ante la incertidumbre — verificar, no inventar—, en vez de asumir que la IA lo va a hacer por defecto. Esto es especialmente importante para datos donde un error es «silencioso»: el sistema no se rompe visiblemente, simplemente deja de funcionar para ese caso específico, algo mucho más difícil de detectar después.

Prompts que necesitaron una vuelta extra de aclaracion

Vale la pena documentar tambien los casos donde la primera version del pedido no fue suficiente, porque ensenan igual o mas que los casos exitosos.

CASO 1 — «LIMITA EL FORMULARIO A EUR» Y DESPUES «NO LO LIMITES»

Cuando se descubrio el bug de monedas, la primera decision fue limitar el formulario a mostrar solo EUR —la unica moneda que el proveedor realmente soporta—. Poco despues, el usuario pidio revertir esa decision, porque el contexto de negocio —los primeros usuarios son chilenos— no habia quedado claro en el momento de tomar esa decision tecnica.

Leccion: una decision tecnicamente correcta puede ser incorrecta para el negocio si no se conoce el contexto de quienes son los usuarios reales.

CASO 2 — EL MENSAJE DE BIENVENIDA POR WHATSAPP, EN VARIAS VUELTAS

Primero se pidio agregar una confirmacion de bienvenida por WhatsApp. Despues, al notar que se veia «poco profesional», se pidio agregar el logo. Despues, al notar que el email tampoco mostraba el logo aunque el codigo ya lo soportaba, se investigo y se encontro que era un problema de configuracion —faltaba una variable de entorno en el lugar correcto—, no de codigo.

Leccion: cuando algo «no se ve como se penso», no siempre es un bug de logica. Muchas veces es una variable de configuracion que existe en un lugar pero falta en otro lugar donde tambien se necesita —este patron se repitio tres veces en el proyecto, ver Capitulo 07.

Como pedir codigo

1. Di el estado actual antes de pedir el cambio. No asumas que la IA recuerda perfectamente todo el codigo de memoria.
2. Especifica que archivos no deben cambiar, si eso importa. Frases como «no cambies nada de la logica de Duffel» o «no toques Supabase» evitaron que cambios pequenos tuvieran efectos secundarios no deseados.
3. Pide que se pruebe, no solo que se escriba. «Pruebalo en local» o «verifica que funcione antes de subirlo» fueron instrucciones explicitas, repetidas en casi todos los pedidos de cambios de codigo.
4. Si el cambio es de tamano medio o grande, pide el plan primero.

Como pedir arquitectura o decisiones de diseno

- Da el contexto de negocio, no solo el tecnico. «Quienes son mis usuarios» importa tanto como «que hace la funcion».
- Si hay mas de una opcion razonable, es valido pedir que la IA presente las opciones con sus tradeoffs, en vez de que decida sola.
- Cuando la decision tiene consecuencias que tu puedes medir mejor que la IA —por ejemplo, «cuantas personas puedo invitar sin pasarme de los limites gratuitos»—, pide que la IA te de los numeros concretos para que tu tomes la decision final.

Como pedir ayuda cuando no sabes por donde empezar

Un patron que se repitio muchas veces, especialmente al principio del proceso de configurar servicios externos —Supabase, SendGrid, Twilio, Duffel—: el usuario no sabia navegar la interfaz de un servicio que nunca habia usado, y en vez de intentar adivinar, pedia guia paso a paso, y cuando algo no calzaba con lo esperado, reportaba exactamente que estaba viendo —a veces con una captura de pantalla—, en vez de intentar seguir adivinando por su cuenta.

HABILIDAD SUBESTIMADA

Reportar con precision lo que ves en pantalla —incluso pegando el texto exacto o una imagen— le da a la IA la informacion que necesita para corregir el rumbo de inmediato, en vez de que ambos sigan una ruta equivocada durante varios pasos mas.

Como dividir proyectos grandes en pedidos mas chicos

El prompt inicial de este proyecto ya venia dividido en seis pasos numerados y ordenados. Esto es una tecnica de division de trabajo excelente: en vez de pedir «haz que mi app funcione en produccion» —una sola instruccion gigante y vaga—, se dividio en piezas independientes, cada una con un criterio de exito verificable por separado.

Regla practica: cada pieza deberia poder verificarse como «funciona» o «no funciona» de forma independiente, sin depender de que las demas piezas tambien esten terminadas.

Como evitar respuestas malas o genericas

- Evita preguntas abiertas sin contexto —«¿como hago un login?»—. Da el contexto completo: que framework usas, que ya existe, que problema específico tienes.
- Si la respuesta se siente genérica, agrega más detalle concreto —archivos, datos reales, mensajes de error exactos— y vuelve a preguntar.
- Cuando algo falla y no sabes por qué, pega el error exacto —el texto completo, no un resumen tuyo de lo que crees que dice.

Reglas de oro para conversar correctamente con una IA

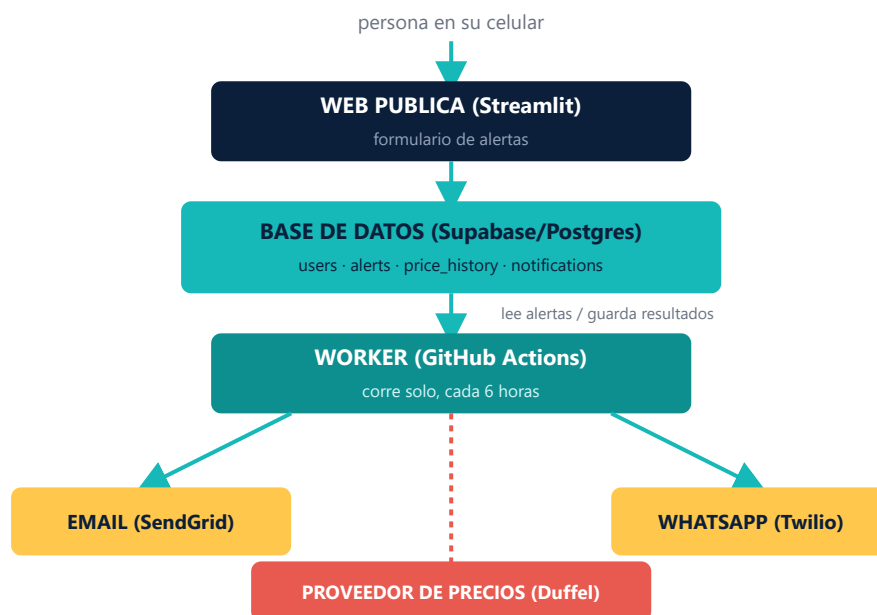
Resumen

- ☐ Da el estado actual antes que el pedido.
- ☐ Da el objetivo final, no solo el siguiente paso.
- ☐ Si hay una decisión de negocio detrás de una decisión técnica, explícala.
- ☐ Pide verificación con datos o pruebas reales, no confirmación teórica.
- ☐ Reporta errores con el texto exacto, no un resumen interpretado.
- ☐ Para cambios grandes, pide plan antes que ejecución.
- ☐ Para datos críticos, pide verificación explícita en vez de asumir memoria perfecta.
- ☐ Si algo no te gusta de una decisión ya tomada, dilo directamente con el contexto que faltaba.
- ☐ Cuando termines una tarea grande, pide que quede documentada.
- ☐ No tengas miedo de pedir explicaciones además de pedir ejecución.

04 Arquitectura del proyecto

Explicada para quien no sabe que es un «backend» o un «hosting». Construimos el vocabulario desde cero, usando Somos-Rata como ejemplo constante.

El mapa general del sistema



Tres piezas independientes que solo se hablan a traves de la base de datos. La web recibe alertas; el worker las revisa por su cuenta.

Vocabulario basico, sin asumir nada

FRONTEND

La parte de un sistema que el usuario ve y con la que interactua directamente: botones, formularios, textos, colores. En Somos-Rata, el frontend es la pagina web hecha con Streamlit —un formulario donde escribes tu ruta, fechas y precio maximo.

BACKEND

La parte de un sistema que no ve el usuario: la logica que procesa datos, se comunica con la base de datos, aplica reglas de negocio. En Somos-Rata, el backend esta repartido en dos piezas: el codigo que procesa el formulario cuando lo envias —dentro de la misma app de Streamlit—, y el worker que revisa precios cada seis horas —un programa separado—.

BASE DE DATOS

Un sistema especializado en guardar informacion de forma organizada y poder buscarla rapido despues. Se organiza en «tablas» —como hojas de Excel, con columnas fijas—, y cada fila es un registro. Somos-Rata usa PostgreSQL, a traves del servicio Supabase, que da un Postgres real, en la nube, gratis para proyectos chicos, con una interfaz visual para verlo sin saber SQL.

HOSTING

El servicio que hace que tu codigo, que hasta ahora solo corria en tu computador, este disponible en internet para cualquiera, con una direccion —URL— publica. Somos-Rata usa Streamlit Community Cloud como hosting de la web, gratis, especializado en apps hechas con la libreria Streamlit de Python.

VARIABLES DE ENTORNO

Datos de configuracion —contrasenas, claves de acceso a servicios, direcciones de servidores— que no se escriben directamente dentro del codigo, sino que se guardan por fuera, en un lugar seguro, y el codigo los «lee» en el momento de ejecutarse.

POR QUE IMPORTA TANTO

Por dos razones: **seguridad** —si escribes una clave secreta directo en el codigo y subes ese codigo a un repositorio, cualquiera con acceso al repositorio ve la clave—; y **flexibilidad** —el mismo codigo puede comportarse distinto en distintos ambientes solo cambiando estas variables, sin tocar el codigo—. En Somos-Rata hay tres lugares distintos donde estas variables pueden vivir, y esto genero mas de un bug real (ver Capitulo 07): tu archivo `.env` local, los Secrets de Streamlit Cloud, y los Secrets de GitHub Actions.

API —APPLICATION PROGRAMMING INTERFACE—

Una forma estandarizada en la que un programa le «pide cosas» a otro programa, generalmente a traves de internet. Cuando el worker de Somos-Rata quiere saber el precio de un vuelo, le hace una peticion al proveedor Duffel, en un formato que ambos entienden, y Duffel le responde con los datos en ese mismo formato.

CRON, WORKER, JOB PROGRAMADO

Un programa que se ejecuta automaticamente en un horario fijo, sin que nadie lo dispare manualmente. En Somos-Rata, el worker —`check_prices.py`— esta configurado para correr cada seis horas, usando GitHub Actions como el servicio que se encarga de «despertarlo» en el horario correcto.

DESPLIEGUE —DEPLOY—

El proceso de tomar tu código y ponerlo a correr en el hosting real, para que deje de ser «algo que solo funciona en mi computador» y pase a ser «algo que funciona en internet, para cualquiera». En Somos-Rata, el despliegue de la web es automático: cada vez que se sube un cambio a GitHub, Streamlit Cloud lo detecta y actualiza la versión pública sola, en uno o dos minutos.

REPOSITORIO, GIT, GITHUB

Git es una herramienta que guarda el historial completo de cambios de tu código: cada versión anterior queda guardada, se puede volver atrás, se puede ver exactamente qué cambio y cuándo. GitHub es un servicio que aloja ese historial en la nube —un «repositorio» o «repo»—, y además permite conectar ese código con otros servicios, como Streamlit Cloud.

El patrón más importante de todo el proyecto: el patrón Strategy

Este es el patrón de diseño que más vale la pena que te lleves de este proyecto, porque resuelve un problema que vas a tener en cualquier proyecto futuro: como desarrollar y probar tu sistema sin depender de servicios externos reales —que cuestan dinero, requieren registro, o simplemente no quieres configurar todavía—, pero de forma que activar la versión real después sea trivial.

La idea, explicada simple: en vez de que tu código llame directamente a «el proveedor de precios de vuelos» o «el proveedor de emails», el código llama a una función genérica —por ejemplo, «buscar vuelos» o «mandar email»—, y esa función genérica decide, según la configuración, a qué implementación específica delegar el trabajo real.

BUSCAR_VUELOS(RUTA, FECHA)

```
si FLIGHT_PROVIDER = "mock"      → genera datos falsos, gratis, instantaneo, para
desarrollar.
si FLIGHT_PROVIDER = "duffel" → llama a la API real de Duffel.
si FLIGHT_PROVIDER = "amadeus" → llama a la API real de Amadeus.
```

Lo mismo aplica para la base de datos —con credenciales de Supabase, guarda en Postgres real; sin ellas, guarda en un archivo CSV local— y para las notificaciones —con API key de SendGrid, manda el email real; sin ella, entra en «modo dry-run» y solo escribe en el registro de actividad lo que habría mandado, sin fallar ni gastar cuota—.

Por qué esto es tan valioso:

1. Puedes desarrollar y probar todo el flujo del sistema sin gastar dinero ni necesitar internet, usando datos simulados.

2. El día que activas el servicio real, no reescribes nada: solo agregas la credencial correspondiente como variable de entorno, y el mismo código empieza a usar el servicio real automáticamente.
3. Si un servicio real falla temporalmente o cambia sus condiciones —como paso con Amadeus en este proyecto—, puedes cambiar de proveedor sin tocar el resto del sistema.
4. Reduce mucho el miedo a «romper todo»: como el modo simulado siempre está disponible, puedes volver a él en cualquier momento para depurar un problema.

En términos de programación, este patrón tiene un nombre formal: patrón de diseño «Strategy». No necesitas memorizar el nombre, pero si necesitas recordar la idea: separa «que se necesita hacer» de «cómo se hace específicamente», y decide cuál implementación usar mediante configuración externa, no mediante cambios de código.

Las cuatro tablas de la base de datos

Somos-Rata guarda toda su información en cuatro tablas, relacionadas entre sí:

Tabla	Que guarda
<code>users</code>	Quien eres: nombre, email, whatsapp, consentimiento.
<code>alerts</code>	Que buscas: origen, destino, fechas, pasajeros, precio máximo, moneda, preferencias, estado.
<code>price_history</code>	Cada precio encontrado en cada revisión, dispere o no una notificación.
<code>notifications</code>	Cada aviso mandado: canal, éxito o fallo, precio al momento del envío.

Por que cuatro tablas y no una sola tabla gigante con todo mezclado: porque cada tabla representa un concepto distinto con su propio ciclo de vida. Un usuario puede tener cinco alertas. Una alerta puede tener doscientos registros de precio histórico —uno por cada revisión del worker durante meses— pero solo tres notificaciones —porque el precio bueno solo apareció tres veces—. Esta forma de organizar datos relacionados en tablas separadas, conectadas por identificadores, se llama **modelo relacional**, y es la base de cómo funciona casi cualquier sistema con una base de datos SQL como Postgres.

La separación entre la web y el worker

Una decisión de arquitectura central: la web publica y el worker que revisa precios y notifica son dos programas completamente independientes, que ni siquiera corren en el mismo lugar —la web en Streamlit Cloud; el worker en GitHub Actions—. Lo único que comparten es la base de datos.

Por que separarlos, en vez de que la web misma revise los precios cuando alguien la visita:

1. Revisar precios de una alerta puede tardar varios segundos. Si eso pasara cada vez que alguien visita la pagina, la pagina se sentiria lenta.
2. Las alertas necesitan revisarse periodicamente, no solo cuando alguien esta mirando la pagina —de hecho, la gracia del producto es que el usuario no tiene que estar mirando la pagina para recibir su aviso.
3. Si el worker falla —por ejemplo, el proveedor de precios esta caido—, eso no debe afectar para nada que la web siga funcionando y la gente pueda seguir creando alertas nuevas.

Esta separacion entre «la parte que atiende al usuario en el momento» y «la parte que trabaja en segundo plano, en su propio horario» es un patron extremadamente comun en sistemas reales —se suele llamar arquitectura de *jobs asincronos* o *background workers*—, y vale la pena replicarlo en cualquier proyecto futuro donde tengas una tarea que tarda tiempo, o que tiene que repetirse periodicamente sin que el usuario la dispare cada vez.

Como se protegen los secretos

Regla de oro, sin excepciones: ninguna clave, contrasena, o token de acceso se escribe directamente en el codigo que se sube a GitHub. Todo eso vive en variables de entorno, y los archivos que las contienen estan explicitamente excluidos del control de versiones mediante un archivo `.gitignore`.

Adicionalmente, se tomo una decision de seguridad consciente al momento de compartir el link publico con usuarios reales: el panel de administracion se protegio con una clave simple antes de invitar gente de afuera, porque hasta ese momento era accesible para cualquiera que entrara al link publico y navegara al panel.

LECCION GENERAL

Antes de compartir un link publico con extranos, revisa todas las paginas o rutas de tu sistema, no solo la principal, para confirmar que ninguna expone informacion que no deberia ser publica.

05 Como trabajamos con Claude Code

Probablemente el capítulo mas importante para quien va a repetir este proceso. Aca se explica la mecanica concreta de colaboracion, no solo la teoria.

Claude Chat versus Claude Code: la diferencia que hay que entender bien

Claude Chat —o cualquier IA en modo conversacion pura, como una web de chat—: es una conversacion de ideas. La IA puede explicarte conceptos, ayudarte a pensar una arquitectura, escribirte un fragmento de codigo de ejemplo para que tu lo copies y pegues. Pero no tiene acceso directo a tu computador ni a tus archivos reales: todo lo que «hace» queda dentro de la conversacion, y eres tu quien tiene que llevarlo a la practica.

Claude Code: es un asistente que corre dentro de tu propio entorno de trabajo —en este caso, conectado directamente a la carpeta del proyecto en tu computador—, con capacidad de:

- Leer y escribir archivos reales de tu proyecto.
- Ejecutar comandos en tu terminal —instalar programas, correr scripts, hacer consultas a bases de datos.
- Hacer commits y subir cambios a GitHub.
- Navegar y usar herramientas externas —buscar en internet para verificar datos.
- Todo esto bajo tu supervision: te avisa que va a hacer, y para acciones riesgosas pide confirmacion antes de proceder.

En esta conversacion real, Claude Code hizo todo lo siguiente de forma directa, sin que el usuario tuviera que copiar y pegar codigo manualmente:

- Instalo Git y GitHub CLI en el computador del usuario, que no los tenia instalados.
- Creo el repositorio en GitHub y subio el codigo.
- Escribio y modifiko decenas de archivos de codigo Python.
- Corrio pruebas reales contra la base de datos de Supabase —crear registros de prueba y despues borrarlos.
- Envio emails y mensajes de WhatsApp reales de prueba, usando las credenciales configuradas.
- Disparo manualmente el worker desde GitHub Actions para verificar su comportamiento sin esperar el horario programado.

- Investigo documentacion externa buscando en internet cuando hizo falta verificar informacion tecnica.
- Genere paginas web de reporte para presentar informacion de forma visual.

REGLA PRACTICA

Usa un chat de IA normal cuando estas en fase de **exploracion de ideas** —que arquitectura conviene, que decision de producto tomar, entender un concepto—. Usa Claude Code —o una herramienta equivalente con acceso directo a tu proyecto— cuando estas en fase de **ejecucion** —escribir codigo real, conectar servicios reales, desplegar.

Como dividir tareas dentro de una sesion

Un patron visible constantemente: cuando una tarea tenia varios pasos claros, Claude Code armaba una lista de tareas visible, y la iba marcando paso a paso a medida que avanzaba, en vez de intentar hacer todo de una vez sin que quede rastro del progreso. Por ejemplo, para la funcionalidad de «buscar por nombre de ciudad», la lista de tareas fue:

1. Crear el archivo nuevo con el diccionario de ciudades.
2. Actualizar el formulario web para usar el nuevo diccionario.
3. Probar la resolucion de ciudades con casos concretos.
4. Levantar la app en local y verificar visualmente.
5. Correr el worker completo para confirmar que nada se rompio.
6. Hacer commit y push.

Esto tiene varias ventajas practicas: siempre sabes en que paso esta el trabajo sin tener que preguntar «¿como vas?»; si la sesion se interrumpe, es facil retomar exactamente donde quedo; y fuerza a pensar el trabajo en pasos verificables uno por uno, en vez de un bloque monolitico que solo se prueba al final.

Como se hicieron los commits, y por que importa

Un **commit** es un punto guardado en el historial de tu codigo, con un mensaje que explica que cambio y por que. Durante todo este proyecto, el patron seguido fue: hacer el cambio, probarlo —localmente, y a veces contra servicios reales—, y recien despues de confirmar que funciona, hacer el commit con un mensaje claro y especifico, y subirlo a GitHub, lo cual dispara automaticamente el redespigüe.

Ejemplos reales de mensajes de commit usados en este proyecto —fijate en que son especificos, no genericos como «cambios» o «fix»—:

Fix critico: no notificar cuando la moneda de la oferta no coincide con la de la alerta; limitar formulario a EUR

Fix: agregar BRAND_LOGO_URL al env del workflow (el secret existia pero nunca se inyectaba)

Buscador por nombre de ciudad: traducir Origen/Destino a codigo IATA automaticamente

REGLA PRACTICA

Nunca hagas commit de algo que no probaste. En este proyecto, la secuencia siempre fue codigo → prueba → commit → push, nunca al reves.

Como se reviso el codigo: verificacion, no confianza ciega

Un principio aplicado de forma consistente: «el codigo compila» o «el codigo se ve bien» nunca fue suficiente para dar un cambio por terminado. Los niveles de verificacion usados, de mas basico a mas riguroso:

Nivel	Que verifica
1	Sintaxis: que el archivo no tenga errores de escritura basicos.
2	Pruebas unitarias sueltas: llamar a la funcion nueva con casos conocidos.
3	Prueba end-to-end contra el sistema real: crear un dato de prueba real y despues borrarlo.
4	Prueba del flujo completo: correr el worker y leer los logs linea por linea.
5	Confirmacion humana: pedirle a una persona real que confirme si le lleo.

Este ultimo nivel fue clave para detectar el bug de BRAND_LOGO_URL no inyectado en el workflow — ver Capitulo 07—: el sistema no arrojaba ningun error, el email se enviaba «exitosamente» segun los logs, pero visualmente le faltaba el logo. Sin la confirmacion humana de «esto se ve simple, no tiene la imagen», ese bug habria quedado sin detectar indefinidamente, porque tecnicamente «no fallaba».

Como iterar sobre una funcionalidad ya construida

El patron tipico de iteracion observado repetidas veces: se construye una version inicial, se prueba con un caso real, el humano da feedback especifico de que no le convence — «se ve poco profesional», «no incluye la imagen»—, la IA ajusta la version sin descartar lo anterior, se vuelve a probar, se repite hasta que el resultado convence.

Esto contrasta con un patron menos eficiente que hay que evitar: pedir «una version perfecta» de entrada, sin iterar. Es mucho mas rapido y efectivo construir una primera version simple, probarla con algo real, y refinar en base a lo que se observa, que intentar anticipar toda la version final sin haber visto nada funcionando todavia.

Como se pidieron mejoras sin romper lo que ya funcionaba

Frases usadas repetidamente en este proyecto, que vale la pena adoptar como habito:

```
«No cambies la logica principal ni rompas lo que ya funciona.»  
«No toques Supabase / Duffel / el cron / las credenciales.»  
«Antes de modificar: revisa la estructura del proyecto.»
```

Estas instrucciones funcionan como un «perimetro de seguridad» alrededor de las partes del sistema que ya estan probadas y funcionando. Ademas, cada vez que se hizo un cambio de tamano medio, se termino con una verificacion explicita de que el resto del sistema seguia funcionando —correr el worker completo y revisar que todas las alertas, no solo la que motivo el cambio, se procesaran correctamente—. Esta es la version practica de lo que en programacion profesional se llama **prueba de regresion**: confirmar que un cambio nuevo no rompio algo viejo.

Como se mantuvo el proyecto limpio

- Los datos de prueba creados durante el desarrollo se borran de la base de datos real inmediatamente despues de confirmar que la prueba funciono.
- Los archivos de configuracion local que no debian subirse a GitHub se verificaron explicitamente que estuvieran excluidos antes del primer commit.
- Cuando se encontro una carpeta de configuracion de la propia herramienta de IA mezclada en el proyecto, se agrego al `.gitignore` para no subirla, manteniendo la separacion entre configuracion de herramientas de trabajo y codigo del producto.
- La documentacion tecnica del proyecto se fue actualizando a medida que pasaban cosas relevantes, no se dejo para «documentar todo al final».

El uso de preguntas de eleccion multiple

Una herramienta especifica que se uso repetidamente merece explicacion propia: cuando habia una decision con dos a cuatro opciones razonables y era el humano quien debia decidir, en vez de escribir un parrafo largo pidiendo una decision en texto libre, se presentaron las opciones de forma estructurada, con una breve descripcion de cada una y sus consecuencias. Ejemplos reales:

- «¿Que key de Supabase usamos, anon o service_role?»
- «¿Instalamos Git con winget, con GitHub Desktop, o ya lo tienes instalado?»
- «¿El repositorio en GitHub debe ser publico o privado?»
- «¿Como seguimos con el tema de la moneda en Duffel: crear equipo nuevo en CLP, o dejarlo en EUR por ahora?»

Este patron de **presentar opciones concretas con sus tradeoffs, en vez de preguntar de forma abierta** acelera mucho la toma de decisiones, porque reduce la carga cognitiva de tener que pensar la respuesta desde cero. Si en el futuro trabajas con una IA que no ofrece este mecanismo de forma nativa, puedes replicarlo tu mismo: cuando tengas que preguntar algo importante, ofrece dos o tres opciones concretas con sus consecuencias.

06 Desarrollo del MVP: la historia completa

La cronología real, con las decisiones, los descartes y los cambios de rumbo. Es el «diario de a bordo» del proyecto.

Punto de partida

El proyecto ya existía como MVP funcional en modo demo: Streamlit + Python, corriendo en local, con un proveedor de vuelos simulado —*mock*— y persistencia en archivos CSV. El código ya tenía una arquitectura pensada para escalar a producción sin reescritura —el patrón Strategy del Capítulo 04—, lo cual fue decisivo para la velocidad de todo lo que vino después.

El pedido inicial fue un plan de seis pasos, en orden explícito: Supabase, GitHub, Streamlit Community Cloud, SendGrid, Amadeus —entorno test—, y Twilio WhatsApp al final.

● PASO 1 · SUPABASE

Se creó el proyecto en Supabase —tier gratuito—, se ejecutó el DDL de las cuatro tablas, y se configuraron las credenciales. Se eligió la *anon key* —clave pública, más simple— en vez de la *service_role key*, dado que solo el propio backend usa esa clave. **Primer bug del proyecto:** al intentar insertar un usuario de prueba, error de «tabla no encontrada» —el DDL nunca se había ejecutado efectivamente. Una vez ejecutado, la conexión funcionó al primer intento.

● PASO 2 · GITHUB

El computador no tenía Git ni GitHub CLI instalados. Se instalaron ambos, se configuró la identidad de Git, se inicializó el repositorio, y se creó el repositorio remoto como **privado**. El primer intento de subir el código falló porque el token de autenticación no tenía el permiso específico para archivos de flujo de trabajo —se amplió el permiso y se reintentó con éxito.

● PASO 3 · STREAMLIT COMMUNITY CLOUD

Se conectó el repositorio —privado— a Streamlit Cloud, otorgando acceso solo a ese repositorio. Primer problema: el link redirigía a un login —la visibilidad estaba en modo restringido, se corrigió a «pública y buscable»—. **Segundo problema, más serio:** al crear una alerta real, error `invalid input syntax for type timestamp`. Este es el primer bug de fondo real del proyecto —ver Capítulo 07, Error 1—: se corrigió convirtiendo cadenas vacías a NULL antes de insertar en Supabase. Con eso, se verificó por primera vez el flujo completo de punta a punta.

PASO 4 · SENDGRID

Primer tropiezo de navegacion: se entro por error al asistente de «Autenticacion de Dominio» en vez de al flujo de «Single Sender». Corregida la navegacion, se verifico el remitente y se creo la API Key. **Bug encontrado:** error 403, «the from address does not match a verified Sender Identity». Causa: un error de tipeo en el correo remitente —«gmial.com» en vez de «gmail.com»—. Corregido el typo, el envio funciono de inmediato.

PASO 5 · EL PIVOTE DE AMADEUS A DUFFEL

El capitulo mas instructivo de toda la cronologia. Al intentar registrar una cuenta en Amadeus for Developers, se encontro un aviso: el portal self-service para desarrolladores seria decomisionado en pocos dias, y el link de registro devolvía «Page does not exist». En vez de insistir, se pivoteo a Duffel —que ya tenia una implementacion preparada en el codigo, gracias al patron Strategy—. Se verifico una busqueda real: mas de cincuenta ofertas devueltas, con aerolineas reales. El worker, con el proveedor real conectado, encontro un precio por encima del umbral y correctamente decidio no notificar. **Bonus:** ni Duffel ni Amadeus exponen un link publico de compra por oferta; se agrego un enlace de referencia a Google Flights.

PASO 6 · TWILIO WHATSAPP

Se creo la cuenta —plan Trial, con credito de prueba—, y se activo el Sandbox de WhatsApp, que exige que cada numero destinatario mande un mensaje especifico —«join palabra-clave»— antes de poder recibir mensajes. Se verifico el flujo dual —email y WhatsApp simultaneo— disparando el worker con una alerta de umbral alto, y ambos canales funcionaron.

BRANDING · DE «FLIGHT PRICE ALERT WEB» A «SOMOS-RATA»

Con el sistema tecnico funcionando de punta a punta, se invirtio en identidad de marca: nombre, eslogan, logo, paleta de colores, aplicados en la web, el email —con tabla de datos del vuelo— y WhatsApp —con imagen de logo adjunta—. Se habilito GitHub Pages para servir el logo de forma publica sin exponer el resto del codigo —el repositorio es privado—. En esta fase se descubrieron y corrigieron dos bugs de «variable de entorno que existe pero no llega a donde hace falta» —ver Capitulo 07—. Antes de compartir el link, se detecto que el panel de administracion quedaria accesible sin proteccion, y se activo una clave de acceso antes de invitar gente externa.

EL BETA CON USUARIOS REALES

Antes de invitar gente, se analizaron los límites de los planes gratuitos de cada servicio, concluyendo que un grupo inicial de diez a treinta personas era completamente seguro. El mensaje de invitación fue iterando varias veces a medida que aparecían problemas reales: primero simple, después con la explicación del orden correcto del «join» de WhatsApp, después con la aclaración de que solo la moneda EUR realmente encuentra coincidencias. Durante el beta se resolvieron, en vivo, con usuarios reales esperando respuesta, múltiples casos de confirmaciones que no llegaban —diagnosticados revisando los logs reales de Twilio, con causas distintas cada vez—.

LA FUNCIONALIDAD NACIDA DEL FEEDBACK: BUSCAR POR CIUDAD

El feedback más repetido de los primeros usuarios fue: «no sé el código IATA de tres letras de mi ciudad». Esto motivó una funcionalidad nueva, fuera del plan original, construida siguiendo el patrón de «plan antes de código» —el usuario ya había investigado que Duffel acepta códigos de ciudad, no solo de aeropuerto específico—. Se resolvieron tres decisiones de diseño mediante preguntas de elección múltiple, se aprobó el plan, y se implementó con pruebas antes de subir el cambio. Poco después, se amplió el diccionario de ciudades a pedido del negocio, y se aplicó rigurosamente la instrucción de «verificar antes de inventar»: se encontró y corrigió un código de aeropuerto desactualizado —ver Capítulo 07, Error 11—.

Resumen de la línea de tiempo

Cuando	Que paso
Día 1	Punto de partida, plan de 6 pasos.
Día 1–2	Supabase + GitHub + Streamlit Cloud + primer bug de producción corregido.
Día 2–3	SendGrid conectado, Amadeus descartado y reemplazado por Duffel, Twilio conectado.
Día 3	Branding completo, dos bugs de secrets corregidos, seguridad activada antes de invitar usuarios.
Día 3–4	Beta con usuarios reales, bug crítico de monedas descubierto y corregido.
Día 4–5	Busqueda por ciudad construida a partir de feedback real, con verificación de datos.

Menos de una semana completa, de «funciona en mi computador con datos falsos» a «producto en producción, con usuarios reales, con una funcionalidad nueva agregada en base a su feedback».

07 Errores que cometimos

Si solo vas a leer un capítulo de este libro antes de tu próximo proyecto, que sea este. Cada error de acá fue real, paso en producción —algunos con usuarios reales esperando una respuesta—, y cada uno enseña algo que se puede prevenir la próxima vez. Formato de cada error: que paso, por que paso —causa raíz—, como se detecto, como se corrigio, y la leccion generalizable.

Errores tecnicos: bugs de codigo y de datos

ERROR 1

NULL versus cadena vacia en Postgres

QUE PASO

La primerísima vez que se probó crear una alerta desde la web pública real, ya conectada a Supabase, el sistema respondió con un error técnico: `invalid input syntax for type timestamp with time zone: ''`.

POR QUE PASO —CAUSA RAIZ

El código original fue diseñado para funcionar tanto con un backend simple de archivos CSV como con Postgres real. En CSV, una cadena de texto vacía es una forma perfectamente válida de decir «este campo no tiene valor todavía». Pero Postgres es mucho más estricto con los tipos de datos: una columna de «fecha y hora» no acepta una cadena vacía, solo acepta una fecha real o el valor especial NULL.

COMO SE DETECTO

Probando el flujo real en la web pública, no revisando el código de forma abstracta. El error solo aparecía al intentar guardar datos reales en la base de datos real.

COMO SE CORRIGIO

Se agrego una función que, solo cuando el backend activo es Postgres/Supabase, convierte cualquier cadena vacía a NULL antes de intentar guardar el registro.

La leccion: cuando tu sistema habla con más de un tipo de almacenamiento, no asumas que un valor válido en un lado lo es en el otro. Los «valores vacíos» —null, cadena vacía, cero, indefinido— son una fuente clásica de bugs sutiles precisamente porque cada sistema representa «no hay dato acá» a su manera.

ERROR 2

Typo en el correo remitente de SendGrid

QUE PASO

Al intentar enviar el primer email real de prueba, SendGrid devolvio un error 403: «the from address does not match a verified Sender Identity».

POR QUE PASO

Un simple error de tipeo humano: «gmial.com» en vez de «gmail.com». El sistema no tenia forma de detectar esto automaticamente porque es, gramaticalmente, un dominio tan valido como cualquier otro.

COMO SE DETECTO

Capturando el mensaje de error exacto y completo de la API de SendGrid, y revisando despues el valor literal guardado, letra por letra.

COMO SE CORRIGIO

Corregir el typo y volver a probar. El envio funciono al primer intento despues.

La leccion: cuando algo falla, obten el mensaje de error completo y exacto, no una interpretacion tuya. Los datos escritos por humanos —correos, dominios, claves— son una fuente de errores tontos pero frecuentes.

ERROR 3

Pipe de terminal que rompía secrets con saltos de línea

QUE PASO

Al configurar varios secrets de GitHub Actions usando una tubería de comandos en la terminal, el sistema fallaba después con «URL inválida» para un dato que visualmente se veía correcto.

POR QUE PASO

El método usado agregaba, de forma invisible, un salto de línea al final de cada valor. Ese carácter invisible convertía una URL válida en un texto técnicamente distinto, aunque a simple vista se veía idéntico.

COMO SE DETECTO

Revisando el registro de actividad del sistema que fallaba, que mostraba explícitamente «No se pudo iniciar Supabase (Invalid URL)».

COMO SE CORRIGIO

Se cambió el método de configurar los secrets, pasando el valor de forma directa, sin la tubería que agregaba el carácter invisible.

La lección: los caracteres invisibles —saltos de línea sobrantes, espacios al final— son muy difíciles de ver a simple vista. Cuando un dato «se ve bien» pero el sistema lo rechaza, sospecha de caracteres invisibles.

ERROR 4 — EL MAS GRAVE DEL PROYECTO

Comparar precios en distintas monedas sin conversion

QUE PASO

Usuarios reales crearon alertas en pesos chilenos, con valores como «500.000» o «2.000.000». El sistema, al revisar precios reales —que el proveedor siempre devuelve en euros, sin importar que moneda pidiera el usuario—, comparaba el numero crudo del precio en euros —por ejemplo, 700— contra el numero crudo del precio objetivo en pesos —por ejemplo, 700.000—, y como 700 es matematicamente menor que 700.000, el sistema «creia» que el precio cumplia la condicion y estaba a punto de mandar notificaciones falsas.

POR QUE PASO —CAUSA RAIZ

El proveedor de precios, en el entorno de pruebas usado, siempre devuelve los precios en euros, sin importar que moneda se le pida —esto depende de una configuracion de la cuenta, fija desde su creacion, no de cada busqueda individual—. El codigo de comparacion de precios nunca verificaba que la moneda del precio encontrado coincidiera con la moneda del precio objetivo del usuario.

COMO SE DETECTO

El propio fundador, revisando los datos reales que estaban ingresando los usuarios, noto un patron sospechoso —un valor de 500 en pesos chilenos para un vuelo internacional es evidentemente un malentendido— y pregunto directamente si el buscador estaba tomando bien los valores en CLP.

COMO SE CORRIGIO

En dos partes. Primero, una correccion de emergencia: se pausaron manualmente todas las alertas en riesgo antes de arreglar el codigo. Segundo, una correccion definitiva: se agrego una regla que compara la moneda de la oferta contra la moneda de la alerta, y si no coinciden, el sistema nunca notifica.

La leccion —probablemente la mas importante de todo el libro: cuando compares dos numeros que representan una medida del mundo real —dinero, distancia, tiempo, peso—, nunca asumas que estan en la misma unidad solo porque ambos son numeros. Verifica explicitamente la unidad antes de comparar, y si no coinciden, la opcion segura por defecto es NO actuar. Un falso positivo en un sistema que le habla directamente a usuarios reales es mucho mas costoso en confianza que «el sistema no hizo nada esta vez».

ERROR 5 — UNA FAMILIA DE TRES CASOS

Secrets que existen pero no llegan a donde hace falta

Este merece explicarse como familia, porque el sintoma fue identico las tres veces —«algo deberia funcionar segun el codigo, pero en la practica no pasa nada, sin ningun error visible»—, pero la causa tecnica especifica fue distinta cada vez.

CASO A — EL WHATSAPP DE BIENVENIDA NUNCA LLEGABA

Las credenciales de Twilio estaban configuradas en los Secrets de GitHub Actions —que usa el worker—, pero nunca se habian configurado en los Secrets de Streamlit Cloud —que usa la web publica—, porque hasta ese momento la web nunca habia necesitado mandar WhatsApp directamente.

CASO B — EL LOGO NO APARECIA EN EL EMAIL

GitHub Actions tiene dos conceptos distintos: tener un secret guardado en el repositorio, y que ese secret se inyecte como variable disponible durante la ejecucion de un flujo de trabajo especifico. El archivo YAML del cron nunca listaba la variable del logo entre las que se inyectan, aunque el secret si existia guardado.

CASO C — LA VENTANA DE 24 HORAS DE WHATSAPP

No es exactamente un bug de configuracion, sino una regla de negocio del propio proveedor —WhatsApp/Meta— que no se conocia de antemano: los mensajes iniciados por el sistema solo se entregan si el usuario escribio algo en las ultimas 24 horas.

COMO SE DETECTARON LOS CASOS A Y B

Gracias a que un humano revisó el resultado real —¿me llego el mensaje? ¿tiene la imagen?—, no solo confio en que «el codigo no arrojó ningun error, entonces debe haber funcionado».

COMO SE CORRIGIERON

Agregando las variables faltantes en el lugar especifico que faltaba en cada caso, y ademas, en el Caso A, agregando registro de actividad explicito del resultado del envio.

La leccion: en cualquier sistema que corre en mas de un lugar —tu computador local, un servidor de hosting, un worker en otro servicio—, cada variable de configuracion nueva tiene que revisarse contra **todos** los lugares donde el codigo que la usa realmente se ejecuta, no solo el lugar mas obvio.

ERROR 6

La terminal corrompiendo caracteres con tilde en una tubería

QUE PASO

Al correr una prueba automatizada que incluía el texto «São Paulo», el resultado mostraba «S??o Paulo», sugiriendo —incorrectamente— que la lógica de normalización de texto tenía un bug.

POR QUE PASO

El problema no estaba en el código de normalización, sino en cómo se estaba ejecutando la prueba: una combinación de comandos de terminal que, en el sistema operativo usado, no preserva correctamente los caracteres especiales durante esa transferencia.

COMO SE DETECTO

Ejecutando la misma prueba de una forma distinta —el archivo directamente, en vez de mediante la tubería—, el resultado fue correcto, lo cual aisló el problema al método de ejecución.

COMO SE CORRIGIO

Cambiar la forma de ejecutar la prueba, sin tocar ni una línea del código real.

La lección: cuando una prueba automatizada falla, la primera pregunta no debería ser «¿qué tiene mal mi código?», sino «¿estoy seguro de que el entorno de prueba está reproduciendo fielmente las condiciones reales?».

Errores de arquitectura y decisiones de proveedor

ERROR 7

Apostar por un proveedor sin verificar su continuidad

QUE PASO

El código, desde antes de esta conversación, ya tenía una implementación preparada específicamente para Amadeus, siguiendo el plan original. Al llegar el momento de activarlo, se descubrió que el portal de registro gratuito estaba a días de cerrarse permanentemente.

POR QUE PASO —A NIVEL DE PROCESO, NO DE CÓDIGO

No fue un error de programación, fue un error de planificación: se asumió que un proveedor externo seguiría disponible en las mismas condiciones, sin verificar su estado actual antes de comprometerse a usarlo en el momento de la implementación real.

COMO SE DETECTO

Al intentar registrarse, apareció un aviso explícito del propio proveedor anunciando el cierre del servicio, junto con un link de registro roto.

COMO SE CORRIGIO

No se intentó «salvar» la decisión original a toda costa. Se evaluó una alternativa —Duffel— que ya tenía también una implementación preparada en el código, y se dejó la decisión final en manos del fundador, con toda la información sobre la mesa.

La lección: antes de comprometerte fuertemente con un proveedor externo —sobre todo uno gratuito o de nivel «hobby»—, verifica su estado actual, no confíes en información de hace semanas o meses. Y cuando pase —va a pasar, tarde o temprano, con algún proveedor de tu stack—, la respuesta correcta no es pelear para salvar la decisión original, es evaluar rápido las alternativas y pivotar con la menor fricción posible.

ERROR 8

No revisar todas las paginas antes de compartir un link publico

QUE PASO

Cuando se estaba por invitar a las primeras personas externas, se identifico que el panel de administracion era accesible para cualquiera que visitara el link publico y navegara hasta esa pagina, sin ninguna proteccion.

POR QUE PASO

Esta pagina se habia construido en una etapa temprana, pensada como herramienta de uso interno, en un momento en que el producto ni siquiera tenia un link publico compartido con nadie.

COMO SE DETECTO

Mediante una revision explicita, antes de compartir el link ampliamente, con la pregunta «¿que le falta para poder compartir esto tranquilamente?».

COMO SE CORRIGIO

Se activo una proteccion por clave para esa pagina especifica —el codigo ya tenia la logica lista, simplemente estaba desactivada— antes de invitar a nadie externo.

La leccion: cada vez que tu proyecto pase de «privado, solo yo lo uso» a «publico, cualquiera puede entrar», haz una revision explicita de todas las paginas de tu sistema, preguntando para cada una: « ¿esta pagina deberia ser visible para un desconocido?».

Errores de producto, UX y comunicacion con usuarios

ERROR 9

Pedir a los usuarios que hicieran algo en el orden incorrecto

QUE PASO

El mensaje inicial compartido con los primeros testers no especificaba un orden entre dos acciones necesarias —unirse al sandbox de WhatsApp, y crear la alerta—. Varios usuarios completaron el formulario primero, y la confirmación automática les fallaba por la ventana de 24 horas.

POR QUE PASO

Una suposición implícita de que «el orden no debería importar», cuando en realidad, por una restricción técnica del proveedor externo, sí importaba mucho.

COMO SE DETECTO

Usuarios reales reportando que «no me llegó nada», cruzado con los logs reales de Twilio.

COMO SE CORRIGIO

Se reescribió el mensaje de instrucciones especificando explícitamente el orden correcto, con una explicación honesta de que si se hace mal «no pasa nada grave», solo hay que repetir el paso.

La lección: cuando le pidas a un usuario que haga varios pasos, y el orden importa por alguna restricción técnica invisible para él, dilo explícitamente, no asumas que es intuitivo.

ERROR 10

Limitar una funcionalidad sin conocer el contexto real de los usuarios

QUE PASO

Al descubrir el bug de monedas, la primera reaccion tecnica fue restringir el formulario para que solo aceptara EUR. Poco despues, el propio fundador pidio revertir esa decision, explicando que los primeros usuarios eran chilenos y pensar sus precios en euros no era natural.

POR QUE PASO

Una decision tecnicamente «correcta» se tomo sin suficiente contexto de negocio, lo cual la convirtio en una decision de producto equivocada, aunque tecnicamente prolija.

COMO SE DETECTO

El propio fundador, al ver el cambio ya aplicado, identifico que no reflejaba la realidad de sus usuarios.

COMO SE CORRIGIO

Se revirtio el limite, y se resolvio el problema de fondo por un camino distinto: comunicando la limitacion directamente a los usuarios afectados, mientras la proteccion tecnica contra falsos positivos seguia activa de todos modos.

La leccion: una solucion tecnicamente prolija no siempre es la mejor decision de producto. Cuando la respuesta tecnica y la de negocio entran en tension, frecuentemente la solucion correcta es separar las dos preocupaciones: dejar la experiencia del usuario como corresponde a su contexto real, y resolver la limitacion tecnica por un camino que no la sienta directamente.

ERROR 11

Dar por sentado un dato sin verificarlo —el código de Castro

QUE PASO

Al pedir ampliar el diccionario de ciudades, el propio fundador sugirió un código específico para Castro, Chile: WCA. Ese código, aunque real, corresponde a un aeropuerto —Gamboa— que ya no opera vuelos comerciales regulares; el aeropuerto realmente usado hoy es otro, con código MHC —Mocopulli—.

POR QUE PASO

Información desactualizada en la fuente original de ese dato específico —el propio fundador, de memoria—. Es exactamente el tipo de error fácil de cometer con datos de nicho que cambian con el tiempo.

COMO SE DETECTO

Porque se aplicó la instrucción explícita, dada de antemano, de «verificar cada código IATA... si dudas de alguno, avísame en vez de inventarlo». Se investigó cada código contra fuentes externas independientes antes de escribirlo en el código.

COMO SE CORRIGIO

Se usó el código correcto y vigente —MHC—, explicando claramente al fundador por qué se cambiaba respecto a lo que el mismo había propuesto.

La lección: para datos críticos —aquellos donde un valor incorrecto rompe silenciosamente una funcionalidad—, no confíes ciegamente ni en tu propia memoria ni en la de una IA. Pide explícitamente verificación contra una fuente externa antes de comprometer el dato al código.

Resumen: los patrones que se repiten en casi todos estos errores

Si agrupas los once errores de arriba, casi todos caen en un número chico de patrones recurrentes, que vale la pena memorizar porque se van a repetir en tus próximos proyectos, aunque el detalle técnico específico sea distinto:

Patrón	Prevención
A. El dato se ve bien pero no lo es (2, 3, 6, 11)	Verificar programáticamente o contra una fuente externa, no solo mirar a simple vista.

B. Funciona en un contexto pero no en otro (1, 5)	Probar contra el sistema real de destino; mantener explicita la lista de «que variable, en que lugares».
C. Comparar cosas que no son comparables (4)	Verificar la unidad/tipo antes de comparar, nunca asumir.
D. Depender de algo externo sin verificar continuidad (7)	Verificar el estado actual antes de comprometerse; diseñar para poder cambiar de proveedor.
E. No revisar el sistema completo al cambiar su alcance (8)	Checklist explícito de seguridad antes de cada cambio de alcance.
F. Suponer que el usuario sabe algo que no sabe (9)	Instrucciones explícitas, sin asumir intuición sobre restricciones invisibles.
G. Decision técnica sin contexto de negocio (10)	Confirmar que conoces el contexto real de quien va a usarlo antes de restringir.

08 Habilidades desarrolladas

Quince habilidades que se entrenaron de forma concreta construyendo Somos-Rata, no de forma teorica. Que es cada una, donde se uso, y como seguir entrenandola.

✓ Pensar en MVP

La capacidad de distinguir «lo minimo necesario para que el producto resuelva el problema completo» de «todo lo que seria lindo tener». Se entreno al resistir la tentacion de construir cosas como conversion de monedas completa antes de tener el producto basico funcionando. **Para seguir entrenandola:** cada vez que se te ocurra una funcionalidad nueva, preguntate si el producto deja de resolver el problema central sin ella.

✓ Prompt engineering

La habilidad de darle a una IA el contexto, la claridad y el nivel de detalle correctos para obtener resultados utiles a la primera. Mejoro visiblemente entre el principio del proyecto y el final. **Para seguir entrenandola:** revisa tus propios prompts despues de una conversacion, identifica en cuales tuviste que aclarar algo despues.

✓ Arquitectura de software

Entender como se organizan las piezas de un sistema y por que se separan de la forma en que se separan. Se entreno con el patron Strategy, la separacion web/worker, y el modelo de cuatro tablas. **Para seguir entrenandola:** antes de tu proximo proyecto, dibuja el mapa de piezas antes de escribir codigo.

✓ UX y comunicacion de producto

Pensar la experiencia desde el punto de vista de alguien sin ningun contexto previo del sistema. Se entreno reescribiendo varias veces las instrucciones para los primeros testers. **Para seguir entrenandola:** lee tus instrucciones como si no supieras nada del sistema.

✓ Deploy y operaciones

Llevar código desde tu computador hasta un sistema real que funciona en internet, de forma repetible y automática. Se entrenó configurando el despliegue continuo de Streamlit Cloud y el cron de GitHub Actions. **Para seguir entrenándola:** intenta que el despliegue sea automático desde el día uno.

✓ Git y control de versiones

Guardar el historial de cambios de tu código, con mensajes claros. Se entrenó instalando Git desde cero y haciendo decenas de commits, cada uno con un mensaje específico. **Para seguir entrenándola:** nunca hagas un commit con mensaje genérico.

✓ Trabajar con APIs y servicios externos

Conectar tu sistema a servicios de terceros mediante credenciales y llamadas programadas. Se entrenó conectando cinco servicios distintos. **Para seguir entrenándola:** lee la documentación sobre límites del plan gratuito antes de dar por sentado como funciona.

✓ Testing —probar antes de confiar

Verificar que algo funciona de verdad, con datos y sistemas reales, en vez de asumir. Se entrenó en los cinco niveles de verificación del Capítulo 05. **Para seguir entrenándola:** antes de dar un cambio por terminado, pregúntate si lo probaste con el sistema real.

✓ Debug —causa raíz, no síntoma

Cuando algo falla, encontrar por qué falla realmente. Se entrenó intensamente en el Capítulo 07: cada uno de los once errores involucró un proceso de investigación antes de aplicar una corrección. **Para seguir entrenándola:** consigue el mensaje de error exacto antes de preguntar «¿cómo lo arreglo?».

✓ Diseño de producto —branding

Darle a un producto técnico una identidad reconocible y coherente, aplicada de forma consistente en todos los canales. Se entrenó construyendo la identidad Somos-Rata desde cero. **Para seguir entrenándola:** centraliza tu identidad de marca en un solo lugar del código.

✓ Iteracion rapida

Construir una version simple, probarla, recibir senal real, y ajustar en ciclos cortos. Se entreno en practicamente cada funcionalidad del proyecto. **Para seguir entrenandola:** muestra la version mas simple de algo lo antes posible.

✓ Recibir y procesar feedback

Escuchar lo que dicen —o lo que hacen— los usuarios reales y convertirlo en cambios concretos. El feedback de «no se el codigo IATA» se convirtio en una funcionalidad completa. **Para seguir entrenandola:** cuando un usuario reporte algo, revisa tambien los datos que genero.

✓ IA aplicada a un flujo de trabajo real

La meta-habilidad que envuelve a todas las demas: saber cuando y como usar una IA como socio de trabajo dentro de un proceso real de construccion de producto. **Para seguir entrenandola:** identifica en que fase estas —exploracion o ejecucion— y elige la herramienta apropiada.

✓ Seguridad basica de aplicaciones

Entender los riesgos minimos de tener un producto publico en internet. Se entreno excluyendo credenciales del control de versiones y protegiendo el panel de administracion antes de invitar usuarios. **Para seguir entrenandola:** antes de compartir cualquier proyecto, haz una pasada explicita de seguridad.

✓ Gestion de recursos y limites

Entender los limites de los servicios gratuitos y tomar decisiones informadas sobre cuanta gente invitar. Se entreno analizando los limites de los cinco servicios usados antes de invitar a los primeros testers. **Para seguir entrenandola:** revisa los limites de cada servicio gratuito antes de compartir con mas de un puñado de personas.

09 Framework reutilizable

Todo el metodo condensado en un framework accionable. Este es el capitulo que deberias releer cada vez que arranques un proyecto nuevo.

El framework en una pagina: 9 fases

FASE 0 — PROBLEMA

Formula el problema en una frase —quien, que le falta, que consecuencia tiene—. Confirma que puedes explicarlo en cinco segundos. Decide el nivel de validacion necesario.

FASE 1 — ALCANCE DEL MVP

Lista todo lo que se te ocurra. Clasifica cada item: MVP, Deseable, o Descartado. Prioriza dentro del MVP por dependencia y por reduccion de incertidumbre.

FASE 2 — ARQUITECTURA MINIMA

Dibuja el mapa de piezas antes de escribir codigo. Disena cada conexion a un servicio externo con el patron Strategy. Decide el modelo de datos.

FASE 3 — CONSTRUCCION EN MODO DEMO

Construye y prueba todo el flujo end-to-end con datos simulados, sin gastar dinero ni depender de servicios externos todavia.

FASE 4 — CONEXION A SERVICIOS REALES, UNO POR UNO

Ordena los servicios por dependencia e impacto. Para cada uno: crea la cuenta, verifica limites del plan gratuito, conecta credenciales, prueba con un caso real, y recien entonces avanza al siguiente. Si un proveedor falla, evalua alternativas rapido.

FASE 5 — REVISION DE SEGURIDAD ANTES DE COMPARTIR

Revisa todas las paginas de tu sistema. Confirma que ninguna credencial esta en el codigo subido. Protege cualquier panel interno. Calcula cuanta gente puedes invitar sin riesgo.

FASE 6 — IDENTIDAD Y COMUNICACION

Recien ahora, con el sistema tecnico funcionando de punta a punta, invierte en nombre, colores, tono, logo. Centraliza la identidad en un solo lugar del codigo. Escribe las instrucciones para tus primeros usuarios pensando en alguien sin ningun contexto previo.

FASE 7 — BETA CON USUARIOS REALES

Invita a un grupo chico primero —diez a treinta personas es un buen punto de partida. Monitorea activamente. Corrige errores en vivo con la misma disciplina de causa raiz del Capitulo 07.

FASE 8 — ITERAR SOBRE FEEDBACK REAL

El feedback repetido es una senal fuerte de que hay algo real que hacer. Vuelve a la Fase 1 para decidir si la mejora entra al «MVP siguiente» o se pospone. Para cambios de tamano medio o grande, pide o haz un plan explicito antes de ejecutar.

FASE 9 — DOCUMENTAR MIENTRAS PASA, NO DESPUES

Cada bug real corregido, cada decision de arquitectura tomada, anotala en el momento, no la dejes para «documentar todo al final».

Como saber en que fase estas

Pregunta de diagnostico rapido:

«¿Puedo, hoy mismo, mostrarle este producto a un desconocido y que lo use de principio a fin, sin que yo tenga que explicarle nada por fuera de la pantalla?»

- Si la respuesta es no porque falta conectar algo tecnico: estas en Fase 3–4.
- Si la respuesta es no por riesgos de seguridad sin resolver: estas en Fase 5.
- Si la respuesta es si pero se ve poco profesional: estas en Fase 6.
- Si ya se ve bien: estas en Fase 7, andate a conseguir tus primeros usuarios reales.
- Si ya tienes usuarios reales usando el producto: estas en Fase 8, tu trabajo ahora es escuchar e iterar.

Diez principios transversales, que aplican a todas las fases

1. Modo simulado primero, modo real despues, mismo codigo para ambos.
2. Nunca des algo por funcionando sin probarlo con el sistema real.

3. Cuando algo falle, busca la causa raiz, no solo el sintoma.
4. Las variables de configuracion viven en varios lugares; verificalas en todos los que aplican.
5. Nunca compares datos sin verificar que estan en la misma unidad o tipo.
6. Para datos criticos, verifica contra una fuente antes de usarlos.
7. Antes de compartir un link publico, haz una revision de seguridad explicita de todo el sistema.
8. La decision tecnica «correcta» no siempre es la decision de producto correcta; conoce el contexto de tus usuarios reales.
9. Escribe instrucciones para usuarios pensando en alguien sin ningun contexto previo.
10. Documenta los bugs y decisiones a medida que pasan, no al final.

Plantilla de prompt inicial para un proyecto nuevo

Basado en el prompt real que arranque este proyecto, aca tienes una plantilla reutilizable:

```
«Este es mi proyecto [NOMBRE], un MVP en [TECNOLOGIA] que  
[ESTADO ACTUAL EXACTO: que funciona hoy, en modo demo o real, en local o  
desplegado]. Quiero [OBJETIVO FINAL CLARO Y CONCRETO].
```

```
Ayudame paso a paso en este orden:
```

- ```
(1) [primer servicio/pieza, la que las demas dependen]
(2) [control de versiones, si no existe todavia]
(3) [hosting/despliegue publico]
(4) [notificacion o canal primario]
(5) [fuente de datos externa clave del negocio]
(6) [canales o mejoras secundarias]
```

```
Empecemos por [PRIMER PASO]: guíame para [ACCION ESPECIFICA].»
```

## Señales de alerta de que te desviaste del framework

---

Llevas mas de un dia sin poder mostrarle nada funcionando a nadie: probablemente estas sobre-construyendo antes de validar. Vuelve a la Fase 1 y recorta alcance.

Estas peleando para que un proveedor externo funcione, y ya perdiste mas de una hora en eso: pregunta te si existe una alternativa mas simple o estable, en vez de insistir a toda costa.

Vas a compartir un link publico y no revisaste explicitamente la seguridad: para. Haz la revision de la Fase 5 primero.

Un usuario reporto un problema y tu primer instinto es «debe haberlo usado mal»: revisa los datos reales primero. Frecuentemente hay un bug real detras de lo que parece un error de usuario.

Estas por invitar a mucha gente sin haber revisado los limites de tus servicios gratuitos: hazlo antes, no despues de que algo se rompa.

# 10 Checklist definitivo

---

*Doscientos veintiseis puntos, organizados por etapa, de «tengo una idea» a «tengo usuarios reales». Uso sugerido: copia esta lista al arrancar un proyecto nuevo, y ve tachando. No todos los puntos aplican a todos los proyectos, pero la estructura si es reutilizable.*

## A · Descubrimiento del problema

- ☐ Puedo escribir el problema en una sola frase.
- ☐ La frase incluye QUIEN tiene el problema.
- ☐ La frase incluye QUE LE FALTA o que friccion vive.
- ☐ La frase incluye QUE CONSECUENCIA negativa tiene eso.
- ☐ Puedo explicar el problema a alguien en menos de 5 segundos.
- ☐ Se de donde viene la idea —frustracion propia, ajena, tarea manual, patron «vigilar y avisar», otro.
- ☐ Identifique si este problema me ha pasado a mi mismo mas de una vez.
- ☐ Identifique si he escuchado esta misma queja de mas de una persona, sin que se conozcan entre si.
- ☐ Se que alternativas existen hoy para este problema.
- ☐ Puedo explicar por que esas alternativas no son suficientes.
- ☐ Identifique si mi propuesta cambia el MECANISMO, no solo los datos que muestra.
- ☐ Revise si este problema es de nicho muy especifico o evidente y comun.
- ☐ Anote la fecha en que formule el problema, para referencia futura.
- ☐ Comparti la formulacion del problema con alguien de confianza.
- ☐ Descarte la idea si, al intentar formularla, no logro que tenga sentido en una frase clara.

## B · Validacion

- ☐ Decidi el nivel de validacion necesario segun que tan evidente es el problema.
- ☐ Si el problema es de nicho: identifique 5 a 10 personas reales con quienes hablar antes de construir.
- ☐ Si hable con usuarios potenciales: pregunte como resuelven el problema HOY, no si «usarian mi producto».

- ☐ Busque si existen productos similares ya en el mercado.
- ☐ Si no existen competidores: investigue por que.
- ☐ Defini que senal real, no opinion, confirmaria que la idea vale la pena.
- ☐ Acepte que la validacion mas fuerte llega despues de tener un MVP y usuarios reales.
- ☐ Documente mis hipotesis de validacion por escrito.
- ☐ Revise mi propio sesgo: ¿estoy validando de verdad, o buscando confirmar lo que ya queria hacer?
- ☐ Defini un criterio de «esto no funciona» ademas de uno de exito.
- ☐ Si la idea paso la validacion inicial: pase a la definicion de MVP.
- ☐ Si la idea no paso la validacion: la descarte o la reformule antes de seguir.
- ☐ Revise si el problema tiene una version mas chica que podria validar mas rapido.
- ☐ Identifique quien seria mi primer usuario real —una persona concreta.
- ☐ Confirme que tengo forma de contactar a ese primer usuario cuando el producto este listo.

## C · Definicion de MVP

- ☐ Escribi la lista completa de todo lo que se me ocurre que el producto podria tener.
- ☐ Clasifique cada item en MVP / Deseable / Descartado.
- ☐ Para cada item de «MVP», verifique «¿sin esto, el producto deja de resolver el problema de punta a punta?».
- ☐ Confirme que mi lista de MVP no incluye login/cuentas de usuario «por si acaso».
- ☐ Confirme que no incluye soporte para multiples proveedores de un mismo servicio «por si acaso».
- ☐ Confirme que no incluye un panel de administracion elaborado antes de tener usuarios reales.
- ☐ Ordene los items del MVP por dependencia.
- ☐ Dentro de cada nivel de dependencia, ordene por lo que reduce mas incertidumbre primero.
- ☐ Defini el «camino feliz» completo del usuario.
- ☐ Verifique que cada paso del camino feliz esta cubierto por algun item del MVP.
- ☐ Identifique la regla de negocio central de mi producto.
- ☐ Escribi esa regla de negocio en una frase clara, verificable.
- ☐ Identifique si mi regla de negocio necesita alguna proteccion anti-spam o similar.

- ☐ Revise mi lista de «Deseable» y confirme que de verdad puede esperar.
- ☐ Revise mi lista de «Descartado» y confirme que no tiene relacion directa con el problema central.

## D · Arquitectura y diseno tecnico

- ☐ Dibuje —aunque sea a mano— el mapa de piezas del sistema.
- ☐ Identifique cual es el frontend.
- ☐ Identifique cual es el backend.
- ☐ Decidi si necesito un worker separado.
- ☐ Si necesito un worker: decidi cada cuanto tiempo debe correr.
- ☐ Disene el modelo de datos: que tablas o entidades necesito.
- ☐ Para cada tabla, defini que campos necesita.
- ☐ Defini las relaciones entre tablas.
- ☐ Para cada servicio externo, aplique el patron Strategy.
- ☐ Defini un modo simulado para cada servicio externo.
- ☐ Confirme que el modo simulado y el modo real usan exactamente el mismo contrato de datos.
- ☐ Decidi como se activa el modo real.
- ☐ Identifique todos los servicios externos que voy a necesitar.
- ☐ Para cada servicio externo, hice un benchmark rapido de al menos dos opciones.
- ☐ Elegi cada servicio priorizando costo, simplicidad de integracion, no «la mejor tecnologia del mundo».
- ☐ Defini donde van a vivir las variables de entorno en cada ambiente donde mi codigo corre.
- ☐ Revise que mi arquitectura separe logica de negocio de detalles de un proveedor especifico.
- ☐ Identifique los puntos donde mi sistema podria fallar.
- ☐ Para cada punto de fallo, decidi que deberia pasar.
- ☐ Revise si mi arquitectura evita «romper todo» si un solo servicio falla.

## E · Setup de entorno de desarrollo

- ☐ Instale el lenguaje de programacion o runtime necesario.
- ☐ Instale Git.
- ☐ Configure mi identidad de Git.
- ☐ Instale las herramientas de linea de comandos de los servicios que voy a usar.
- ☐ Cree un archivo de dependencias.
- ☐ Verifique que puedo correr mi proyecto en modo simulado, sin ninguna credencial real, desde el primer dia.
- ☐ Cree un archivo de ejemplo de variables de entorno, sin exponer valores reales.
- ☐ Cree mi archivo real de variables de entorno, separado del de ejemplo.
- ☐ Configure un archivo **.gitignore** ANTES del primer commit.
- ☐ Verifique que el **.gitignore** excluye mi archivo de credenciales real.

## F · Base de datos

- ☐ Cree la cuenta en el proveedor de base de datos elegido.
- ☐ Cree el proyecto/instancia de base de datos.
- ☐ Escribi el codigo de definicion de tablas correspondiente a mi modelo de datos.
- ☐ Ejecute el DDL en el editor SQL del proveedor.
- ☐ Verifique visualmente que las tablas se crearon correctamente.
- ☐ Obtuve las credenciales de conexion.
- ☐ Decidi que nivel de clave usar, segun donde se va a usar esa clave.
- ☐ Guarde las credenciales en mi archivo local.
- ☐ Probe la conexion desde mi codigo, ANTES de seguir avanzando.
- ☐ Probe insertar un registro de prueba real.
- ☐ Probe leer ese registro de vuelta.
- ☐ Borre el registro de prueba despues de confirmar que funciona.
- ☐ Verifique que mi codigo cambia automaticamente de modo simulado a modo real solo por la presencia de las credenciales.
- ☐ Revise los limites del plan gratuito.



- ☐ Anote esos limites en algun lugar donde los pueda consultar despues.

## G · Control de versiones / GitHub

- ☐ Inicialice el repositorio Git localmente.
- ☐ Revise que archivos quedarian incluidos en el primer commit ANTES de confirmarlo.
- ☐ Confirme que ninguna credencial real esta en los archivos a subir.
- ☐ Hice el primer commit, con un mensaje claro.
- ☐ Cree el repositorio remoto en GitHub.
- ☐ Decidi si el repositorio es publico o privado, y por que.
- ☐ Conecte el repositorio local con el remoto.
- ☐ Subi el primer commit al remoto.
- ☐ Verifique en la web de GitHub que el codigo subido es el correcto.
- ☐ Configure los permisos necesarios de mi token/CLI para las acciones que voy a necesitar.
- ☐ Defini una convencion simple de mensajes de commit y la segui de forma consistente.
- ☐ Verifique que cada commit corresponde a un cambio ya probado.

## H · Despliegue / Hosting

- ☐ Elegi el servicio de hosting apropiado para mi stack tecnologico.
- ☐ Conecte el hosting con mi repositorio de GitHub.
- ☐ Si mi repo es privado: otorgue acceso SOLO a ese repositorio especifico.
- ☐ Configure las variables de entorno/secrets en el panel del servicio de hosting.
- ☐ Despliegue la aplicacion por primera vez.
- ☐ Verifique que el link publico carga sin pedir login ni mostrar errores.
- ☐ Revise la configuracion de visibilidad del despliegue.
- ☐ Probe el «camino feliz» completo desde el link publico.
- ☐ Confirme que cada nuevo push redespiega automaticamente la version publica.

- ☐ Guarde el link publico en un lugar accesible para compartir despues.
- ☐ Verifique el comportamiento en un navegador movil.
- ☐ Revise los limites de recursos del plan gratuito de hosting.

## I · Notificaciones — Email

- ☐ Cree la cuenta en el proveedor de email transaccional elegido.
- ☐ Verifique un remitente —correo o dominio, segun mi volumen.
- ☐ Revise letra por letra el correo remitente configurado.
- ☐ Cree la clave de API con el permiso minimo necesario.
- ☐ Guarde la clave de API en mi archivo local.
- ☐ Probe un envio real de prueba a mi propio correo.
- ☐ Revise mi bandeja de entrada, y spam, para confirmar la llegada.
- ☐ Si el correo cae en spam: entienda por que.
- ☐ Configure las mismas credenciales en el lugar de configuracion del worker/cron.
- ☐ Verifique el limite diario/mensual del plan gratuito.
- ☐ Disene el contenido del email pensando en distintos clientes de correo.

## J · Fuente de datos externa / API de negocio

- ☐ Identifique cual es LA fuente de datos externa central de mi negocio.
- ☐ Verifique el estado actual de ese proveedor ANTES de comprometerme fuertemente.
- ☐ Cree la cuenta en el entorno de pruebas/sandbox de ese proveedor.
- ☐ Genere las credenciales de prueba.
- ☐ Lei, aunque sea por encima, la documentacion sobre que formato de datos devuelve.
- ☐ Verifique en que UNIDADES vienen los datos criticos, y si eso es configurable o fijo.
- ☐ Probe una consulta real de ejemplo, y revise la respuesta completa.
- ☐ Identifique si el proveedor tiene una alternativa/competidor por si falla.

- ☐ Confirme que mi arquitectura permite cambiar de proveedor sin reescribir la logica de negocio.
- ☐ Configure las credenciales tambien en el ambiente del worker/cron.
- ☐ Verifique el limite de uso del entorno de pruebas.
- ☐ Documente cualquier limitacion conocida del entorno de pruebas.

## K · Canal secundario de notificacion —ej. WhatsApp/SMS

- ☐ Evalúe si un canal secundario es realmente necesario para el MVP, o puede esperar.
- ☐ Cree la cuenta en el proveedor elegido.
- ☐ Active el entorno de pruebas/sandbox correspondiente.
- ☐ Entendi las restricciones especificas del sandbox.
- ☐ Documente esas restricciones en un lenguaje simple, pensando en como se las voy a explicar a mis usuarios.
- ☐ Probe un envio real de prueba.
- ☐ Confirme la recepcion con una persona real, no solo revisando logs.
- ☐ Configure las credenciales en TODOS los ambientes donde el codigo que las usa realmente corre.
- ☐ Diseñe el mensaje de este canal pensando en que puede tener menos espacio/formato que un email.
- ☐ Revise el costo real antes de asumir que es gratis indefinidamente.

## L · Seguridad antes de compartir publicamente

- ☐ Liste TODAS las paginas/rutas de mi sistema, no solo la principal.
- ☐ Para cada pagina, pregunte «¿deberia ser visible para un desconocido?».
- ☐ Proteja con clave o elimine el acceso publico a cualquier panel interno.
- ☐ Confirme que ninguna credencial esta hardcodeda en el codigo.
- ☐ Confirme que mi `.gitignore` sigue excluyendo todos los archivos sensibles.
- ☐ Revise que la clave de base de datos que uso desde mi backend es la de menor privilegio necesario.
- ☐ Si mi sistema pide datos personales: incluya un texto de consentimiento explicito.
- ☐ Revise si necesito ademas activar seguridad a nivel de base de datos.

- ☐ Confirme que los mensajes de error no exponen detalles tecnicos sensibles.
- ☐ Revise que no estoy haciendo scraping ni usando datos de forma no autorizada.
- ☐ Verifique que mi repositorio no tiene commits antiguos con credenciales expuestas.
- ☐ Defini un plan basico de «que hago si algo se filtra».
- ☐ Revise los terminos de servicio de cada proveedor gratuito que uso.
- ☐ Confirme que tengo forma de desactivar o pausar el sistema rapidamente.
- ☐ Revise que el manejo de datos personales cumple, al menos, con el sentido comun de privacidad.

## M · Branding e identidad

- ☐ Confirme que el sistema tecnico funciona de punta a punta ANTES de invertir tiempo en identidad visual.
- ☐ Defini el nombre del producto.
- ☐ Defini el eslogan o propuesta de valor en una frase.
- ☐ Defini una paleta de colores, nombrada.
- ☐ Obtuve o cree un logo.
- ☐ Centralice nombre, eslogan, colores y logo en UN SOLO lugar del codigo.
- ☐ Aplique la identidad en la interfaz principal.
- ☐ Aplique la identidad en las notificaciones por email.
- ☐ Aplique la identidad en el canal secundario si corresponde.
- ☐ Disene un comportamiento de respaldo para cuando algun recurso de marca no este disponible.
- ☐ Si necesito que un logo sea accesible desde fuera y mi repositorio es privado: resolví como publicar SOLO ese recurso.
- ☐ Probe la identidad de marca con datos/casos reales.

## N · Beta con usuarios reales

- ☐ Calcule cuantas personas puedo invitar sin riesgo, revisando los limites de CADA servicio gratuito.
- ☐ Identifique cual es el limite mas restrictivo de todos.
- ☐ Defini un numero concreto y justificado de personas para la primera ronda.

- ☐ Escribi el mensaje de invitacion pensando en alguien sin NINGUN contexto previo.
- ☐ Si hay pasos con un orden especifico: los explicita claramente.
- ☐ Explique claramente cualquier limitacion temporal conocida del producto.
- ☐ Comparti el link con el primer grupo.
- ☐ Monitoree activamente los primeros usos.
- ☐ Revise los datos reales que se generaron, no solo las quejas explicitas.
- ☐ Prepare un canal directo de comunicacion con los primeros testers.
- ☐ Responde con precision tecnica cuando un usuario pregunte algo, revisando el codigo real si hace falta.

## O · Monitoreo y verificacion continua

- ☐ Se donde ver los logs/actividad de cada servicio externo que uso.
- ☐ Se como forzar una ejecucion manual de mi worker/cron, sin esperar el horario programado.
- ☐ Revise periodicamente el panel de actividad de mi proveedor de email.
- ☐ Revise periodicamente el panel de actividad de mi proveedor de mensajeria.
- ☐ Confirme, con casos reales, que la logica anti-spam de mi sistema funciona como espero.
- ☐ Establezca una rutina de revisar el sistema cada cierto tiempo mientras hay usuarios activos.
- ☐ Verifique que mi base de datos no esta por pausarse/suspenderse por inactividad.

## P · Manejo de errores y bugs

- ☐ Cuando algo falle, obtuve el mensaje de error COMPLETO Y EXACTO antes de intentar corregir nada.
- ☐ Investigue la causa raiz, no solo aplique un arreglo al sintoma.
- ☐ Verifique si el mismo tipo de bug podria estar presente en otro lugar similar.
- ☐ Si el bug tenia riesgo inmediato para usuarios reales: tome una medida de contencion rapida antes de la correccion definitiva.
- ☐ Probe la correccion contra un caso real que reproduzca el bug original.
- ☐ Confirme que la correccion no rompio nada mas —prueba de regresion.
- ☐ Agregue registro de actividad adicional si el bug era dificil de detectar.

- ☐ Documente el bug: que paso, causa raiz, como se corrigio.

## Q · Documentacion

- ☐ Mantengo un archivo de documentacion tecnica del proyecto, separado del codigo.
- ☐ Actualizo esa documentacion a medida que pasan cosas relevantes, no solo al final.
- ☐ Documento las variables de entorno necesarias y donde se configuran.
- ☐ Documento las decisiones de arquitectura importantes y el por que.
- ☐ Mantengo una seccion de «pendientes conocidos».
- ☐ Mantengo una seccion de bugs encontrados y corregidos.

## R · Iteracion y feedback

- ☐ Tengo un canal para recibir feedback de mis usuarios reales.
- ☐ Reviso el feedback buscando PATRONES, no solo casos aislados.
- ☐ Para cada pieza de feedback fuerte, la paso de nuevo por el proceso de definicion de MVP.
- ☐ Para cambios de tamano medio/grande motivados por feedback, hago un plan explicito antes de implementar.
- ☐ Verifico decisiones/datos sugeridos por usuarios contra una fuente externa antes de comprometerlos al codigo, si son datos criticos.
- ☐ Pruebo cada mejora nueva end-to-end antes de darla por lista.
- ☐ Despliego la mejora y confirmo que el redespliegue fue exitoso.
- ☐ Comunico la mejora a los usuarios afectados si corresponde.

## S · Preparacion para escalar / monetizacion

- ☐ Identifique cuales de mis servicios gratuitos tienen un limite que voy a alcanzar si el producto crece.
- ☐ Investigue el costo de pasar a un plan pago en cada uno de esos servicios.
- ☐ Identifique que funcionalidades «Deseables» tienen mas sentido de construir ahora que hay uso real.
- ☐ Revise si necesito mejorar la seguridad antes de escalar el numero de usuarios.
- ☐ Revise si necesito migrar algun canal de «modo sandbox» a «modo produccion aprobado».

- ☐ Piense en como autenticar mejor mis canales de notificación para mejorar entregabilidad.
- ☐ Identifique métricas básicas que quiero empezar a trackear.
- ☐ Piense en un modelo de monetización posible, sin necesariamente implementarlo todavía.
- ☐ Revise que mi arquitectura sigue siendo apropiada al nuevo volumen, o si necesita evolucionar.
- ☐ Considere si necesito ayuda humana adicional a partir de cierto volumen de usuarios.
- ☐ Revise el feedback acumulado de todo el beta para decidir la próxima gran prioridad de desarrollo.
- ☐ Celebre haber llegado hasta acá. Muy pocos proyectos llegan a tener usuarios reales usando algo que construiste de principio a fin.

**TOTAL: 226 PUNTOS**

# 11 Filosofía de trabajo

---

*Un capítulo mas personal y menos operativo que los anteriores: la forma de pensar que sostuvo todo el proceso descrito en este libro.*

## La IA es un multiplicador de criterio, no un reemplazo de criterio

---

La idea mas importante de toda esta filosofía: Claude —o cualquier IA con la que trabajes— no reemplaza tu juicio sobre que construir, para quien, y por que. Multiplica tu capacidad de ejecutar ese juicio. Cuando en este proyecto la IA tomo decisiones tecnicas equivocadas por falta de contexto de negocio —el caso de limitar el formulario a EUR sin saber que los primeros usuarios eran chilenos—, la correccion no vino de «pedirle a la IA que lo pensara mejor» en abstracto: vino de que el humano apporto el contexto que faltaba.

Esa es la division de trabajo correcta: tu aportas el contexto, el objetivo, y el criterio de «esto esta bien / esto no me convence»; la IA aporta velocidad de ejecucion, amplitud de conocimiento tecnico, y capacidad de mantener consistencia en un sistema complejo.

## Velocidad con disciplina, no velocidad sin control

---

Este proyecto se construyo en menos de una semana, de cero a produccion con usuarios reales. Eso podria sonar a «trabajar rapido y sin cuidado», pero es lo opuesto: la velocidad vino de tener una disciplina clara en cada paso —probar antes de dar por terminado, verificar causa raiz de cada error, no comprometer seguridad por apuro—, no de saltarse pasos. De hecho, varios de los momentos mas lentos del proyecto —el bug de moneda, los tres casos de «secret que existe pero no llega a donde hace falta»— habrian sido mucho mas lentos de resolver si no hubiera existido la disciplina de verificar con datos reales en cada paso.

La velocidad real no viene de saltarse verificaciones, viene de tener un ciclo de retroalimentacion CORTO —construir algo chico, probarlo de inmediato, corregir de inmediato—, en vez de un ciclo largo —construir mucho, probar al final, descubrir muchos problemas acumulados a la vez.

## Construir para aprender, no solo para terminar

---



En varios momentos de este proyecto, el objetivo inmediato no era «termina esta funcionalidad» sino «entiende que esta pasando de verdad». El ejemplo mas claro es el bug de monedas: el fundador no se conformo con «el sistema esta mandando alertas raras», investigo hasta entender exactamente por que, y esa comprension profunda fue lo que permitio tomar la decision correcta despues.

Cuando algo no funciona como esperabas, resiste el impulso de solo «hacer que el sintoma desaparezca». Invierte el tiempo en entender el por que. Ese entendimiento se vuelve conocimiento reutilizable, mientras que un parche sin entendimiento frecuentemente vuelve a fallar de otra forma despues.

## **No tenerle miedo a cambiar de rumbo**

---

El caso de Amadeus versus Duffel es el ejemplo mas claro: un plan explicito, acordado desde el principio, se abandono en cuanto la realidad mostro que ya no era viable, sin apego emocional a la decision original. Lo mismo paso, a menor escala, con la decision de limitar el formulario a EUR: se tomo, se revirtio horas despues al recibir mejor contexto, sin que revertir una decision se sintiera como un fracaso.

Esta flexibilidad no es indecision. Es lo opuesto: es tener claro cual es el objetivo real —que el producto funcione bien para los usuarios reales—, y estar dispuesto a cambiar el CAMINO hacia ese objetivo tantas veces como haga falta, sin confundir el camino con el objetivo.

## **La honestidad sobre las limitaciones es mejor que ocultarlas**

---

Cuando se descubrio que el producto, temporalmente, solo podia buscar precios reales en euros, la opcion facil habria sido ocultar esa limitacion. La opcion que se tomo fue la opuesta: comunicar la limitacion directamente a los usuarios afectados, con una explicacion honesta y un pedido concreto, con disculpas cuando corresponde.

Este tipo de honestidad, aunque a veces se sienta incomoda, construye mucha mas confianza a largo plazo que ocultar el problema. Y en un producto chico, con pocos usuarios, la confianza de esos primeros usuarios es uno de los activos mas valiosos que tienes: son las personas que te van a dar el feedback mas honesto y que te pueden recomendar a otros despues.

## **El detalle importa, pero en el momento correcto**

---

Este proyecto invirtio tiempo real en detalles finos: verificar codigos de aeropuerto con fuentes externas, corregir un mensaje de bienvenida para que se sintiera mas calido, simplificar un header de

email para que no se viera redundante. Pero todos esos detalles se atendieron DESPUES de que lo esencial —el sistema funcionando de punta a punta con datos reales— estuviera resuelto, no antes.

La regla implicita que se siguio: el detalle importa, pero solo despues de que lo fundamental funcione. Pulir un mensaje de bienvenida que todavia no le llega a nadie es tiempo mal invertido. Pulir ese mismo mensaje una vez que ya le esta llegando a usuarios reales, es tiempo bien invertido.

## Como no frustrarse cuando algo no funciona

---

En un proyecto de la duracion y complejidad de este —cinco servicios externos conectados, decenas de decisiones, mas de diez bugs reales encontrados—, es normal sentir frustracion cuando algo falla. Algunas actitudes que ayudaron:

- Tratar cada error como informacion util, no como un fracaso personal.
- Separar «esto no funciona todavia» de «esto no va a funcionar nunca».
- Pedir ayuda especifica en vez de quedarse trabado en silencio.

## Lanzar rapido, mejorar constantemente

---

El producto se compartio con los primeros usuarios reales ANTES de que todo fuera perfecto. Esta decision —lanzar con limitaciones conocidas, comunicadas con honestidad, en vez de esperar a la version «completa»— es coherente con todo lo demas que dice este libro: el valor real se aprende con uso real, no con anticipacion perfecta.

## Resumen de la filosofia en una lista corta

---

- La IA multiplica tu criterio, no lo reemplaza.
- Ciclos cortos de retroalimentacion son mas rapidos que ciclos largos «perfectos».
- Cuando algo falla, entiende el por que antes de aplicar el arreglo.
- No te apegues emocionalmente a una decision si la realidad cambio.
- Se honesto con tus usuarios sobre las limitaciones.
- Pule el detalle solo despues de que lo esencial funcione.
- Trata los errores como informacion, no como fracasos.

■ Lanza con lo mínimo viable, honesto sobre sus límites, y mejora con uso real.

# 12 Lecciones finales

---

## Que volveria a hacer exactamente igual

---

- Empezar con un plan de pasos claro y ordenado, en vez de un pedido abierto tipo «hazme una app de vuelos». El plan de seis pasos inicial fue la base de toda la velocidad del proyecto.
- Mantener la arquitectura de «modo simulado / modo real intercambiable» —patron Strategy— desde el principio. Sin ella, el pivote de Amadeus a Duffel habria sido mucho mas costoso.
- Verificar cada pieza con un caso real antes de darla por terminada. Ningun bug importante de este proyecto se habria detectado revisando solo el codigo sin ejecutarlo contra los sistemas reales.
- Compartir el producto con usuarios reales relativamente pronto, y usar su feedback real para priorizar que mejorar despues, en vez de intentar anticipar todo de antemano.
- Documentar los bugs y decisiones a medida que pasaban, no al final. Este mismo libro habria sido mucho mas pobre e impreciso si se hubiera escrito solo de memoria.

## Que haria diferente la proxima vez

---

- Verificar el estado de continuidad de un proveedor externo critico ANTES de comprometerse con el en el plan inicial, no al momento de intentar activarlo.
- Antes de agregar cualquier variable de configuracion nueva, hacer explicitamente la pregunta «¿en que TODOS los lugares donde mi codigo corre, esta variable hace falta?».
- Investigar, ANTES de construir el selector de monedas en el formulario, en que moneda(s) realmente puede responder el proveedor de datos elegido.
- Establecer, desde el dia uno, un habito mas sistematico de probar escribiendo texto con tildes en cualquier prueba automatizada que involucre datos en espanol.
- Revisar la seguridad de TODAS las paginas del sistema un poco mas temprano en el proceso, no justo antes de compartir el link publicamente.

## Que le recomendaria a alguien que hoy quiere crear su primera startup con IA

---

1. Empieza con un problema que TU MISMO entiendas de forma visceral, no uno que suene bien en teoria.
2. No confundas «hablar con una IA sobre tu idea» con «validar tu idea».

3. Aprende a distinguir cuando estas en modo exploracion y cuando estas en modo ejecucion.
4. No le tengas miedo a la parte no-tecnica de construir un producto.
5. Cuando algo falle —y va a fallar—, no lo tomes como senal de que esto no es para ti.
6. Invierte en entender el «por que» detras de cada decision tecnica.
7. Comparte tu producto con gente real ANTES de que se sienta «listo».
8. Se honesto con tus usuarios sobre las limitaciones de tu producto.
9. Documenta mientras construyes, no despues.
10. Vuelve a leer este libro —o el que hagas tu, sobre tu propio proximo proyecto— antes de empezar el siguiente.

## Un cierre

---

Somos-Rata, el proyecto que sirvio de caso de estudio para todo este libro, paso de ser un MVP corriendo en modo demo en un computador, a ser un producto real, en produccion, con base de datos en la nube, precios de vuelos reales, notificaciones reales por email y WhatsApp, identidad de marca propia, y usuarios reales generando alertas y dandote feedback real sobre como mejorarlo, en menos de una semana de trabajo.

Ese ritmo no fue magia ni fue casualidad. Fue el resultado directo de:

- Una arquitectura pensada para poder desarrollar sin depender de servicios externos, y activar la version real sin reescribir nada.
- Una disciplina constante de probar con datos y sistemas reales antes de dar cualquier cosa por terminada.
- Una relacion de trabajo con la IA basada en dar contexto claro, verificar resultados, y mantener el criterio de negocio y producto en manos humanas.
- La voluntad de diagnosticar causas raiz en vez de aplicar parches, y de pivotar rapido cuando la realidad lo exigia.

El proximo proyecto que construyas, sea cual sea el problema que elijas resolver, puede seguir este mismo camino. Ese es, en definitiva, el proposito completo de este libro: que el metodo sobreviva al proyecto especifico que lo genero, y este listo para ti la proxima vez que tengas una idea que valga la pena construir.



## **Fin del libro**

Manual personal de metodo, construido documentando en tiempo real el desarrollo de Somos-Rata — Flight Price Alert Web.

Julio de 2026

